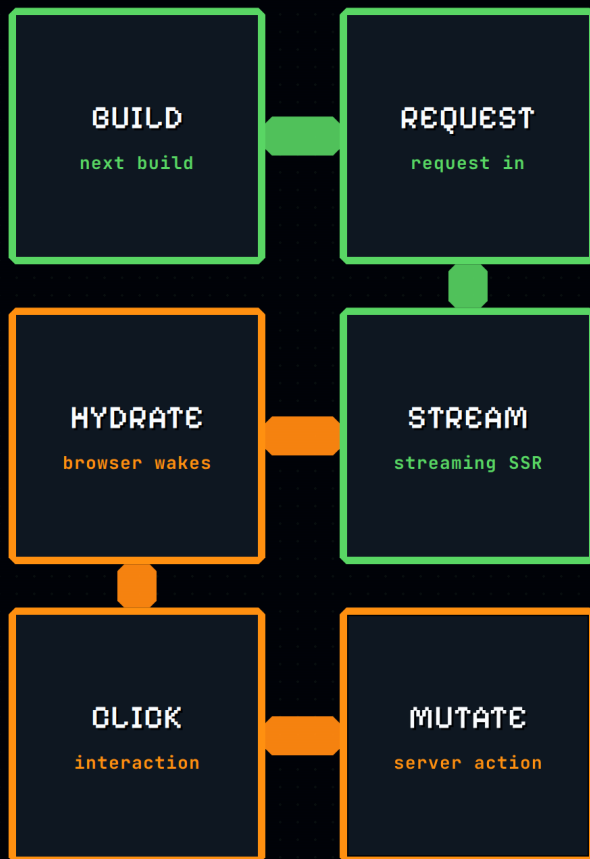


PLAYER ONE

12 LEVELS



WHAT RUNS WHEN

THE MISSING MENTAL MODEL FOR REACT AND NEXT.JS

MARKETING FLUFF



SIGNAL



MAYANK JHUNJHUNWALA

2026 EDITION • NEXT.JS 16 • REACT 19.2

Contents

Before You Start	1
Chapter 1 — Rendering Patterns: A Map for the Genuinely Confused	2
The baseline: what actually happens in a CSR app	2
The two metrics you need	3
SSR: what the server actually sends	4
Wait — isn't Next.js still an SPA?	5
Layouts vs templates	7
So is Next.js an SPA?	7
SSG: do the work once	8
ISR: static speed, periodic freshness	8
What ISR is genuinely wrong for	9
Streaming SSR: don't wait for the slowest thing	10
React Server Components: the bundle size story	11
Static vs dynamic RSC: where SSG lives now	13
Putting it together: a real scenario	13
Two decisions, not four modes	14
The honest summary	15
Chapter 2 — How React Actually Updates the DOM	17
The problem before React	17
What the Virtual DOM actually is	18
How reconciliation works — and why it's $O(n)$, not $O(n^3)$.	19
The key trap — why index breaks things	20
Re-renders — the JS tax React doesn't talk about enough	21
Does reconciliation skip children of changed nodes? .	22
One tree, or two?	22
Is VDOM costlier than direct DOM?	23
memo, useMemo, useCallback — what they actually do	24
The one condition that silently makes all three useless . .	25
Fiber — when React stopped blocking the browser	27
startTransition — telling React what's urgent	27

Hydration mismatch — what actually happens at the reconciler level	28
Signals — a completely different model	30
What you lose without the VDOM	31
Why React hasn't adopted Signals	32
The full picture	32
Chapter 3 — What Runs When, and What Comes Out	34
First, what “render” even means	34
So the server never touches your CSS	35
Why the word is confusing	36
Three artifacts	36
The same file runs at different times	38
Moment 1: next build	38
What actually runs in that pass	41
Moment 2: a request comes in	42
Why the first response contains both	42
Moment 3: the browser wakes up	43
The count	44
But React builds a tree of everything. Doesn't it need the code?	44
Moment 4: the user clicks a link	45
Moment 5: interaction	46
Moment 6: a mutation	46
The whole schedule	47
Back to the four questions	48
Appendix: the one thing that always runs	49
Chapter 4 — What Your Bundler Actually Does	52
First, the graph	52
Why bundling survived native ESM	53
The pipeline	54
Chunking: how the bundler decides what goes in which file	55
Entry points	55
import() is a split point	56
Chunk groups	56
Module IDs, and why they decide your cache hit rate . . .	57
Minification is not tree shaking	58
Your dev bundle is not your prod bundle	59

What changed in Next 16	59
Go and look at your own build	60
References	61
Chapter 5 — Two Graphs, One Repo	62
What "use client" compiles to	62
Two graphs	63
The mechanism: export conditions	64
The manifest	65
Imports across the boundary	65
Server importing client: fine	65
Client importing server: this is where it goes wrong	66
server-only, and how it actually works	66
Where you put the directive decides what you ship	67
The barrel file version	67
Push the boundary down	68
Children, and the bundling reason it works	69
Third-party packages with no directive	69
Go and look	70
References	71
Chapter 6 — Suspense Is Not a Loading Spinner	72
What suspends	72
What actually goes over the wire	73
The same boundary, two transports	74
On a first load	74
On a navigation	75
Two mechanics worth knowing	75
Boundaries are also hydration units	76
What streaming costs you	76
The sibling primitive: error boundaries	77
References	78
Chapter 7 — A Mutation, End to End	79
What "use server" compiles to	79
A detour: what RPC actually means	80
Every action is a public endpoint	82
What travels with the call	83
Forms work before your JavaScript does	84

The response is a tree	85
The hooks	85
Redirects and errors	86
References	88
Chapter 8 — What’s Actually Left for the Client	89
useState is a default, not a decision	89
Owner 1: the server	90
Owner 2: the URL	91
Owner 3: the form	91
Owner 4: ephemeral UI state	92
Where does this state actually live?	93
The audit	93
Two things that aren’t state	94
useReducer, for state that moves together	95
Context is not a state manager	95
Does a context change re-render the whole tree?	96
useSyncExternalStore, the honest escape hatch	97
So what’s actually left?	99
References	100
Chapter 9 — What Redux Was Actually For	101
Misconception 1: “It’s a state manager”	101
What 2015 looked like	101
Misconception 2: “The single store is an arbitrary rule”	102
Misconception 3: “Global state means Redux”	103
Misconception 4: “Reducers are just ceremony”	104
Misconception 5: “Big app, therefore Redux”	105
Misconception 6: “It’s already in the project, so everything goes there”	105
The question to ask first	107
References	107
Chapter 10 — Effects Are Not Lifecycle	108
Where the lifecycle idea came from	108
The cleanup function is not “unmount”	109
The clearest case: <Activity>	109
Which is what StrictMode is for	110
Effects you don’t need	110

The deps array is a correctness contract	112
useEffectEvent, for reading without reacting	112
useLayoutEffect, and the actual sequence	113
“I just want it to run on mount”	114
What RSC deleted	115
Effects that are still correct	116
References	117
Chapter 11 — Caching: Which Layer Is Lying to You	118
Start with what you’ve seen: the page that was already there	118
Go one level down: the page that never hit your code . .	119
“But my content changes. I can’t build it once and freeze it forever.”	120
Go deeper: caching a piece instead of a whole page . . .	122
The one detail that will burn you on deploy	122
The same directive, moved up, does more	123
The bottom layer: the same read, many times, one render	125
The switch that turns caching on — and the error it introduces	126
Now put all four together	128
Invalidation, one story per layer	128
Which layer is lying to you	129
References	130
Chapter 12 — The Map	131
The lifecycle	131
Reading order	133
The pattern underneath	134
Glossary	135
Index	140
About the Author	143

Before You Start

I am not some perfect engineer to be lecturing you. I don't have credentials to flaunt.

I am just someone who loved science growing up, where things are often deterministic. Development and software engineering follow closely in those footsteps, where causality and determinism are core pillars.

AI is coding for us, faster and better than we used to. A few years ago, we had to rely on the community for help. It could be your senior who willingly or unwillingly helped you unblock yourself, or kind folks on the internet, mostly on Stack Overflow. A bigger problem was that you didn't know what you did not know.

AI abstracts it all away. Information is just a prompt away. Software is getting easier to build.

But the problem with an engine of high output is that people are moving away from basics. Not reading docs, not struggling through problems themselves — the kind of struggling that leaves you with core skills. This is very apparent in all the early and mid-level interviews I have been taking. Now that coding isn't your core strength, let's focus more on engineering.

This material is focused on just that. You have worked with these things; you probably have a decent hands-on idea. This book is the next step from there. Clarifying your mental models, stripping away the marketing, bringing clarity.

You could, practically speaking, jump in wherever you want. But I have written it as a series. It does not strictly speaking build on itself chronologically — this sequence is just the one that makes the most sense to me.

I hope this helps at least one person, other than myself.

Chapter 1 — Rendering Patterns: A Map for the Genuinely Confused

Rendering patterns are one of those topics where everyone nods along in standup and then quietly Googles “SSR vs SSG” for the fourth time that week.

I’ve been working with React and Next.js for a while now. And I’ll be honest — these concepts confused me for longer than I’d like to admit. Not because the ideas are hard, but because the industry has a habit of rebranding old ideas with new names, reversing its own opinions every three years, and selling each reversal as a prophecy.

When I started with React, CSR was the answer to everything. Multi-page apps were “legacy.” The Single Page Application was the future. Then SSR came back. Then static generation. Then server components. Then partial pre-rendering.

If you’ve been on this ride, you’re not confused because you’re slow. You’re confused because the goalposts kept moving and nobody gave you a proper map.

This is that map.

We’re going to build it the way I actually understood it — one question at a time, with honest answers.

The baseline: what actually happens in a CSR app

Let’s start with what most React developers learned first.

When a user visits a React SPA for the first time, here’s exactly what happens:

1. The browser makes a request to the server

2. The server responds with a near-empty `index.html` — essentially just `<div id="root"></div>`
3. The browser downloads the JavaScript bundle (or multiple chunks if code-splitting is set up)
4. The JS executes — React calls `root.render()` and builds the DOM from scratch
5. If the app needs data, `useEffect` fires, a fetch request goes out, and a loading spinner appears
6. Data arrives, React re-renders with actual content
7. Dynamically injected images begin loading — this is your LCP event

That step 5 is the one worth staring at. The browser completed three full round trips before the user saw anything meaningful:

HTML loads → JS loads → JS executes
→ data fetch → content appears

This is called the **waterfall problem**. It’s the original sin of pure CSR.

A note on terminology. When React “builds the DOM from scratch” in a CSR app, this is not called hydration. Hydration is a specific term for when React attaches event listeners to existing server-rendered HTML. In a pure CSR app, there’s no server-rendered HTML to hydrate. React is doing a full `root.render()` — building everything from zero.

The two metrics you need

Before going further, two performance metrics matter here:

FCP (First Contentful Paint). When the user first sees anything — a heading, a nav, any pixel that isn’t a blank white screen.

TTI (Time to Interactive). When the page actually responds to user input within 50ms. A page can look complete and still not be interactive if the main thread is busy executing JavaScript.

DOMContentLoaded fires when the browser has finished parsing the HTML and built the DOM tree. CSS and images may still be loading. JS that's deferred has also finished running by this point. It does not mean the page looks good or is interactive.

load fires when everything is done — images, stylesheets, fonts, subresources. This is what `window.onload` uses.

In a CSR app, `DOMContentLoaded` fires almost immediately — on the empty div. But TTI comes much later, after the entire JS bundle has downloaded, parsed, and executed. The gap between these two is the “SPA tax.” On a slow mobile connection with a large bundle, this gap can be 5–8 seconds.

`DOMContentLoaded` (empty div, useless)

- JS downloads + executes
- React renders
- TTI (actually interactive)
- load (images etc done)

SSR: what the server actually sends

Server-Side Rendering flips the model. Instead of sending an empty shell, the server:

1. Receives the request
2. Fetches the data it needs
3. Renders complete HTML with that data already baked in
4. Sends finished HTML to the browser

The browser receives real content and can paint immediately. FCP is fast. SEO crawlers are happy.

But here's what most tutorials gloss over: **the page looks interactive before it is.**

After the HTML renders, the browser still downloads and executes the JavaScript bundle. React then “hydrates” the page — walks the entire DOM tree, reconstructs its Virtual DOM, and attaches event listeners to each node. During this entire process, buttons look clickable. They aren't.

This gap between “looks ready” and “is ready” is called the **hydration gap**.

What happens if you click during the hydration gap? Three outcomes, depending on the element:

- **Dropped** — React hasn't attached a listener yet. The click fires into the void.
- **Replayed** — React captures the event, finishes hydrating, then fires the handler.
- **Double-fired** — a native browser default (form submit, anchor navigation) catches it, and React's synthetic handler fires after hydration. You get the action twice.

Wait — isn't Next.js still an SPA?

After learning SSR, most people assume Next.js works like a traditional multi-page app — every link click goes to the server, which responds with a fresh HTML document, causing a full page reload.

It doesn't. And this is the part that took me an embarrassingly long time to properly understand.

When you use Next.js's `<Link>` component, there is no browser navigation. No request for a new HTML document. No full page reload. The `<Link>` component intercepts the click and handles it with JavaScript — exactly like a SPA would.

You can verify this yourself: open DevTools, go to the Network tab, and click a `<Link>`. Filter by Fetch/XHR. You'll see a request — but it returns a small payload, not a full HTML document.

So what's Next.js actually doing on navigation?

The first time you visit a Next.js site, you get full SSR HTML — great FCP, data already baked in. But the moment that page loads, the Next.js router takes over. From that point on:

1. You click a `<Link>`
2. Next.js intercepts it — no browser navigation
3. It fetches only the data and structure for the new page (more on the format shortly)
4. It patches only the changed part of the DOM
5. The URL updates via the browser's History API

The browser never wipes memory and starts over. Which means — if you have a `<Header>` and `<Footer>` in your `layout.js`, they literally never get destroyed when you navigate. Same DOM nodes. Any state inside them — a search input's current value, a dropdown's open state — survives the navigation.

	Sequence
First load	Server → Full HTML → fast FCP → JS hydrates
Subsequent	JS intercepts click → fetches new page data → patches DOM → no reload

This is the hybrid. SSR for the first load, SPA behaviour for everything after. The industry calls this “Universal” or “Isomorphic” rendering — the same app renders on the server initially and then takes over on the client.

Next 16 extends this further. Instead of unmounting the route you're leaving, it wraps it in React's `<Activity>` component in hidden mode — so component state in the page you navigated away from

survives too. What happens to that page’s *effects* is a separate question with a surprising answer, and Chapter 10 takes it up.

The <a> tag trap. If you use a plain `<a href="/about">` instead of `<Link href="/about">`, you lose all of this. The browser treats it as a real navigation, wipes the page, and requests fresh HTML from the server. Your layout gets destroyed and rebuilt. This is why headers “flicker” or reset in some Next.js apps — someone used an anchor tag where they should have used `Link`.

Layouts vs templates

Next.js makes this persistence explicit:

	Behaviour on navigation	Use it for
layout.js	Persists. Same component instance, state intact	Navigation bars, audio players, cart state, theme toggles
template.js	Remounts. Fresh instance, state reset	Per-page entrance animations, anything that must re-run on every route change

The default is `layout.js`. Reach for `template.js` only when you explicitly want the reset.

So is Next.js an SPA?

In behaviour — yes. In architecture — it’s a hybrid. The correct answer depends on which question you’re asking:

- “Does it do full page reloads?” → No. SPA behaviour.
- “Does the server do rendering work?” → Yes, on first load and for data. Not purely client-rendered.

- “Is the JavaScript bundle the only thing the server sends?” → No. First load is real HTML.

Don’t get too attached to the label. What matters is understanding the two modes: SSR on first hit, SPA on every subsequent click.

SSG: do the work once

Static Site Generation takes SSR’s idea further. Instead of rendering HTML on every request, you render it once at build time and store the result as flat HTML files served from a CDN.

No server processes your request. Response is instant, infinitely scalable, impossible to crash with traffic.

When SSG is right: marketing pages, blog posts, documentation — anything where content doesn’t change per user or per minute.

When SSG is wrong: 10,000 product pages, where build time becomes painful. Data that changes frequently. Anything personalised per user.

ISR: static speed, periodic freshness

Incremental Static Regeneration sits between SSG and SSR. You get static-file delivery speed, but the page refreshes after a time window you control.

The important thing to understand is the exact model — called **stale-while-revalidate**:

Request	What happens
User A hits /product at 12:00	Cache cold → server renders → caches → serves it

Request	What happens
User B hits /product at 12:00:45	Within window → served from cache (stale, fast)
User C hits /product at 12:01:05	Past window → served stale immediately, background rebuild triggered
User D hits /product at 12:01:10	Gets the freshly rebuilt page

Notice: User C gets stale data. ISR doesn’t guarantee freshness *at* the revalidation boundary. It guarantees freshness for the next request *after* revalidation completes.

This is why time-based ISR is often the wrong tool. If your data changes when an admin publishes, use on-demand ISR instead:

```
// Call this from a CMS webhook or Action after a content update
revalidatePath('/blog')
revalidateTag('posts', 'max')
```

What ISR is genuinely wrong for

Scenario	Why ISR fails
User-specific data	ISR caches one version for everyone. User A’s dashboard gets served to User B. That’s a data leak, not a staleness bug.
Auth-gated pages	The same failure with worse consequences — the CDN caches the first authenticated user’s render and serves it to everyone behind it.
Real-time data	You always get the previous snapshot. Use SSR with a WebSocket instead.

Scenario	Why ISR fails
Inventory and transactional	“2 items left in stock” going stale means overselling — a business logic error, not just bad UX.

ISR gained real capability in Next 16 — on-demand generation for routes you never prebuilt, and a shell that can ship before the URL is even known. Chapter 11 covers what changed and why it matters at scale.

Streaming SSR: don't wait for the slowest thing

Regular SSR is all-or-nothing. If your page has three data sources and one is slow, the server waits for all three before sending a single byte.

Streaming SSR breaks this.

With `<Suspense>`, you mark parts of the component tree as “this can wait.” The server immediately flushes everything outside the `Suspense` boundary as chunk one. The browser starts rendering the shell and downloading JS. When the slow data arrives, the server sends chunk two — the rendered component HTML plus a tiny inline script telling React which fallback to replace.

```
export default function Dashboard() {
  return (
    <main>
      { /* renders immediately, sent in chunk 1 */ }
      <Header />
      <Suspense fallback={<Skeleton />}>
        { /* awaits data, sent in chunk 2 when ready */ }
        <SlowWidget />
      </Suspense>
    </main>
  )
}
```

```
    </Suspense>
  </main>
)
}
```

This uses HTTP chunked transfer encoding — not a new protocol. It’s existed since HTTP/1.1. The browser’s HTML parser is incremental by design. It renders what it receives, not what it’s waiting for.

What is `<Suspense>` actually doing mechanically? It’s a signal to the server: “this is a safe boundary to split the response.” Everything outside can be flushed now. Everything inside arrives later, as a separate chunk, over the same open HTTP connection.

Selective hydration works alongside this. React hydrates each Suspense boundary independently as its chunk lands — not the whole page at once. If you click an area that hasn’t hydrated yet, React jumps the queue, hydrates that boundary first, and replays your click.

Chapter 6 takes Suspense apart properly. Almost everything people believe about it is wrong, and the misconceptions all come from that word “fallback.”

React Server Components: the bundle size story

This is the one that trips most React developers up, because it sounds like “component-level SSR.” It isn’t.

SSR produces an HTML string. The browser renders it immediately (fast FCP), then downloads the entire JS bundle to reconstruct React’s Virtual DOM and hydrate. Even a completely

static `<Footer>` — one that never changes, has no event handlers — still requires its JS shipped to the browser so React can account for it during hydration.

RSC changes the output format.

Instead of HTML, the server produces an **RSC payload** — a serialized description of the component tree. Schematically, it looks something like:

```
[ "$", "h1", null, { "children": "Hello World" } ]
```

Server Components never send their JavaScript to the browser. Zero kilobytes. If your `<ProductList>` uses a 200KB library, that library stays on the server. Only the rendered output crosses the network.

On first load: Next.js sends both — HTML for fast FCP and the RSC payload for React to hydrate correctly. They serve different consumers. HTML is for the browser’s native renderer. The payload is for React’s reconciler.

On subsequent `<Link>` navigations: only the RSC payload is sent. The shell is already on screen. React patches only what changed — which is why your layout never re-renders when you navigate between pages. This is what makes Next.js an SPA in behaviour despite doing server rendering.

The most common mistake in Next.js codebases:

putting `'use client'` at the top of a page component because you need one `useState` somewhere inside it. This ships the entire page’s JS to the browser and loses all RSC benefits. The fix: keep the page as a Server Component, extract only the interactive widget into its own Client Component.

Chapter 5 explains why that fix works at the level of the module graph, and why the directive behaves like the root of a subtree rather than a label on one file.

Static vs dynamic RSC: where SSG lives now

In the App Router, you don't "choose RSC or SSG." You write an RSC — the component type — and decide the data strategy, which is when it runs and whether it's cached.

```
// Dynamic RSC – the default. Runs on every request.
async function LiveInventory() {
  const stock = await db.inventory.find(id)
  return <span>{stock} left</span>
}

// Static RSC – opt in explicitly.
async function ProductInfo() {
  'use cache'
  cacheLife('hours')
  const product = await db.products.find(id)
  return <h1>{product.name}</h1>
}
```

Next.js 16 inverted this. Under `cacheComponents`, nothing is cached by default — every page, layout and route handler runs at request time unless you explicitly opt in with the `use cache` directive.

The old model — cached by default, made dynamic by touching `cookies()`, `headers()` or `searchParams` — is what most blog posts still describe. If you learned that model, unlearn it.

The mental model:

RSC = the component type (server-only execution)
SSG/SSR = the delivery strategy (when it runs, cached or not)

Putting it together: a real scenario

A job board with four distinct requirements. What rendering strategy for each?

Homepage — featured jobs, updated a few times a day by an admin. → ISR with on-demand revalidation. Same content for all users, changes infrequently. Trigger `revalidatePath('/')` from your CMS webhook. Static speed, fresh when it matters.

Job detail pages — 10,000 individual listings. → ISR, not SSG. Building 10,000 pages at deploy time is expensive. ISR generates each page on first request and caches it. Use `generateStaticParams` to pre-build only your top 100 most-visited listings at build time; the rest get ISR on demand.

“My Applications” page — per-user, auth-gated. → SSR, dynamic rendering. Personalised and auth-gated. ISR would cache one user’s data and serve it to the next person who visits that route. Must be dynamic, must check the session on every request.

“Who’s viewing this job” counter — live, real-time. → CSR, inside the ISR page. The job detail page is ISR. The counter is a ‘`use client`’ component that mounts after hydration and polls or connects via WebSocket. The page loads at static speed; the counter populates shortly after.

The job detail page uses two strategies simultaneously — ISR for the shell, CSR for the live counter. This is the key insight: **rendering strategies are component-level decisions, not page-level ones.**

Two decisions, not four modes

The names above read as though SSG, ISR, PPR and SSR are four modes you select from a menu. Under Next 16 they aren’t. They’re what emerges from two choices:

1. **Where does cache go?** → decides what’s static
2. **Where do Suspense boundaries go?** → decides what streams

Every name in this chapter is a label for a combination of those two answers. Once you can make both choices deliberately,

the taxonomy stops being something to memorise and becomes something to derive.

The honest summary

The industry spent the 2010s moving everything to the client. It spent the 2020s moving it back to the server — but smarter, with component-level granularity instead of page-level all-or-nothing.

Pattern	What you gain	What you give up
CSR	Simplicity, rich interactivity	FCP, SEO, and you inherit the waterfall problem
SSR	Fast FCP, fresh data on every request	The hydration gap, per-request server cost
SSG	Speed, infinite scalability	Data freshness, and fast builds at scale
ISR	Static speed with periodic freshness	Guaranteed freshness at the revalidation boundary
RSC	Smaller JS bundle, no hydration cost for static parts	useState and browser APIs in that component
Streaming SSR	Progressive rendering, better TTFB	Simplicity — boundaries become a design decision
PPR	Static and dynamic on one page, no manual architecture	Default in 16; you now decide cache boundaries explicitly instead of having them inferred

None of these patterns are universally correct. Each is a tradeoff.

The goal isn't to pick one and defend it. The goal is to understand the tradeoffs well enough to compose them correctly — per page, per component, per use case.

That's the map. The rest of this book is the territory: what actually runs, in what order, and what comes out of each step.

Chapter 2 — How React Actually Updates the DOM

Here's a question I sat with for longer than I should have: if React's Virtual DOM is supposed to make things faster, why does a complex React app sometimes feel slower than a simple jQuery one?

The answer is that the Virtual DOM was never really about raw speed. It was about something more valuable — and understanding what that actually is changes how you think about React, why Signals exist, and what the future of UI frameworks might look like.

Let's start from the beginning.

The problem before React

Before React, building a dynamic UI meant writing imperative code. You found a DOM node, changed it, found another, changed that. Something like:

```
document.getElementById('username').textContent = user.name
document.getElementById('avatar').src = user.avatar
document.getElementById('bio').textContent = user.bio
```

For a small app, this is fine. As the app grows, you're manually tracking which DOM nodes need updating for every possible state transition. You miss one — the UI goes stale. Two developers touch the same node — inconsistent state. The business logic and the DOM update logic become inseparable spaghetti.

The actual problem wasn't performance. It was **correctness and maintainability at scale**.

React’s proposal in 2013 was radical for the time: what if you just re-described the entire UI whenever anything changed, and let the framework figure out what to actually update? The Virtual DOM was the mechanism that made “re-describe everything” cheap enough to do on every state change.

The honest framing. VDOM was sold as a performance innovation. It’s more accurately a developer experience innovation with acceptable performance characteristics. The distinction matters when you’re evaluating alternatives like Signals.

What the Virtual DOM actually is

The Virtual DOM is a plain JavaScript object tree that mirrors the structure of your UI.

When you write JSX:

```
<div className="card">
  <h1>{title}</h1>
  <p>{description}</p>
</div>
```

React doesn’t immediately touch the browser’s DOM. It creates a lightweight JS object:

```
{
  type: 'div',
  props: { className: 'card' },
  children: [
    { type: 'h1', props: {}, children: [title] },
    { type: 'p', props: {}, children: [description] }
  ]
}
```

Creating and comparing these objects is cheap — it's just JavaScript. Touching the real DOM is expensive — it triggers layout calculations, style recalculations, and repaints.

When state changes, React produces a new VDOM tree, compares it against the previous one, and applies only the differences to the real DOM. This comparison process is called **reconciliation**.

Approach	What happens
Old way	Data changes → you manually find and update the right DOM nodes
React	Data changes → re-render entire component (cheap, it's just JS objects) → diff new VDOM against old VDOM → apply only the diff to the real DOM

How reconciliation works — and why it's $O(n)$, not $O(n^3)$

Comparing two arbitrary trees is an $O(n^3)$ problem in computer science. For a tree with 1000 nodes, that's a billion operations. Obviously not viable.

React reduces this to $O(n)$ with two deliberate assumptions.

Assumption 1: different component types produce different trees.

If a `<div>` changes to a `<span>`, React doesn't attempt to reconcile the children. It tears the entire subtree down and builds fresh. React assumes a type change signals a fundamentally different structure — comparing children would be wasted work.

This lets React skip entire branches of the tree without inspecting them.

Assumption 2: the key prop identifies stable elements across renders.

Without keys, React compares list items by position — index 0 to index 0, index 1 to index 1. With keys, React tracks item identity across renders regardless of where in the list that item appears.

This collapses list reconciliation from “try every possible mapping” to “match by identity.”

The key trap — why index breaks things

Indexes are stable when the array itself is stable. The problem only appears when the array mutates. Three cases that break silently:

Reordering. A user sorts the list. The item that was at index 0 is now at index 3. React sees index 0 still exists and assumes it’s the same component instance. Any local state in that component — an expanded accordion, a checked checkbox, a half-typed input — belongs to the position, not the item. It stays at index 0. Your newly sorted item inherits the wrong state.

Inserting at the front or middle. You prepend an item. Everything shifts down by one index. React diffs index 0 changed, index 1 changed, index 2 changed — and patches every node even though only one new item was added.

Deleting from anywhere except the end. Same problem mirrored.

```
// Breaks silently when list mutates
{items.map((item, index) => (
  <TodoItem key={index} text={item.text} />
))}
```

```
// Correct – key travels with the data, not the position
{items.map((item) => (
```

```
<TodoItem key={item.id} text={item.text} />
  )})
```

The dangerous part isn't that it throws an error — it doesn't. It produces subtly wrong UI that's hard to reproduce and harder to trace.

Keep this mechanism in mind. Chapter 10 uses it deliberately, as the correct way to reset state when a prop changes.

Re-renders — the JS tax React doesn't talk about enough

This is the distinction that confuses most React developers, including experienced ones.

Reconciliation operates at two levels.

The render phase (JavaScript). When a component's state changes, React calls that component's function. If the function returns child components, React calls those too — recursively, all the way down the tree. It's re-executing JavaScript to produce a new VDOM.

The commit phase (DOM). React compares the new VDOM against the old one. Only nodes that actually changed get DOM updates.

The commit phase is efficient. The render phase — by default — is not:

Parent state changes

- Parent function re-executes ✓ (expected)
- Child A function re-executes (even though it's unrelated)
- Child B function re-executes (same — unrelated, runs anyway)
- New VDOM produced for entire subtree
- Diff finds nothing changed in Child A or B
- DOM untouched for Child A and B ✓

The DOM is safe. But you ran JavaScript for components that had no reason to re-render. On a dashboard with 500 components, this adds up.

Does reconciliation skip children of changed nodes?

No — and this is the part that trips everyone up.

When a DOM node's attribute changes — a `className`, a style, a text value — React updates that node and leaves its children alone if they haven't changed.

But when a *component* re-renders, its children re-execute their JS regardless, because React has to call their functions to produce new VDOM to diff against. The DOM children might not change, but the JS ran anyway.

So there are two separate questions:

Question	Answer
Did the JS re-execute?	Yes, for the whole subtree by default
Did the DOM get updated?	Only where the diff found actual differences

One tree, or two?

In React, reconciliation uses two trees at the same time, via a technique called **double buffering**.

React does not mutate a single virtual tree in place, nor does it maintain an arbitrary collection of trees. Under React's Fiber architecture, it maintains exactly two Fiber trees:

1. The **current** tree — the Fiber tree currently rendered and visible in the real DOM.

2. The **workInProgress** tree — the Fiber tree React constructs and reconciles in memory during the render phase when a state update occurs.

How double buffering works:

Render / reconciliation phase. When state changes, React clones or creates nodes from the current tree into a new **workInProgress** tree. React computes the diffs, flags changes — insertions, deletions, updates — and prepares the side effects in memory without touching the real DOM. This work can be paused, aborted, or restarted.

Commit phase. Once the **workInProgress** tree is fully calculated, React flushes all changes to the actual DOM in a single synchronous pass.

Pointer swap. After the commit finishes, React simply swaps its root pointer: the **workInProgress** tree becomes the new current tree, and the old tree is repurposed for future updates to avoid excessive garbage collection.

Why not just one tree? If React mutated a single in-place tree, it couldn't pause, split, or abandon rendering work — a core requirement for Concurrent React and transitions — without leaving the UI in a broken, half-updated state. And it would risk rendering *tearing*: inconsistent UI states read during rendering.

Is VDOM costlier than direct DOM?

Think about the cost. Maintaining two VDOM trees, re-running entire subtrees, diffing the new tree against the old, then diffing the result against the real DOM — all to end up doing what one direct DOM write would have done.

If you know exactly which DOM node to change, `element.textContent = 'x'` is faster than React's full cycle.

But that's not the comparison that matters. The real comparison is:

Approach	What it takes
VDOM	Re-render whole component → diff → minimal DOM update
Manual DOM	Track every possible state transition and write individual update logic for each one

At small scale, manual DOM is faster. At application scale, manual DOM becomes unmaintainable and bug-prone. You start missing updates, creating inconsistent state, writing spaghetti imperative code.

VDOM's cost is the price of not having to think about which DOM nodes to update. You just describe the UI; React figures out the rest.

Bottom line: VDOM is a deliberate performance tradeoff that buys you developer productivity and correctness. It's not free.

memo, useMemo, useCallback — what they actually do

Three tools exist to stop unnecessary re-renders. Most developers use them loosely or “just in case.” Here's what each actually does at the reconciler level.

memo wraps a component. Before React calls the component function, it checks: have any props changed since last render? If no — skip the function call entirely. No new VDOM produced for that subtree.

```
const Child = memo(({ name }) => {  
  return <div>{name}</div>  
})
```

This is the only one that actually prevents a re-render. The other two are supporting tools.

useMemo caches a computed value between renders. The component still re-renders — **useMemo** just skips recomputing an expensive calculation if its dependencies haven't changed.

```
const sorted = useMemo(() => {  
  return bigArray.sort(complexComparator)  
}, [bigArray])
```

useCallback caches a function reference between renders. On its own it does almost nothing useful. Its only real job is to support **memo**.

```
const handleClick = useCallback(() => {  
  doSomething(id)  
}, [id])
```

The mental model:

memo	→ the goal (prevent re-render)
useMemo	→ stabilises values so memo 's check succeeds
useCallback	→ stabilises functions so memo 's check succeeds

The one condition that silently makes all three useless

memo compares props with `Object.is()` — essentially `===`. Primitives compare by value. Objects, arrays, and functions compare by reference.

A new reference equals a changed prop, as far as **memo** is concerned — even if the contents are identical.

```
// This memo never works  
const Child = memo(({ config }) => <div>{config.title}</div>)
```

```
function Parent() {  
  // New object every render = new reference = memo always fails  
  return <Child config={{ title: 'hello' }} />  
}
```

Every render of `Parent` creates a new config object. `memo` sees a new reference, assumes the prop changed, re-renders `Child`. You wrote `memo` and got nothing.

Same trap with functions:

```
// New function reference every render = memo fails  
return <Child onClick={() => doSomething()} />
```

```
// Fix – stabilise the reference with useCallback  
const handleClick = useCallback(() => doSomething(), [])  
return <Child onClick={handleClick} />
```

This reference equality problem is what makes memoization fragile. Miss one unstable prop and the whole thing fails silently. Which is exactly what React Compiler is designed to eliminate — it statically analyses your components at build time and injects stable references automatically.

If you're on Next.js 16 with React Compiler enabled, you mostly write plain functions and objects and the compiler handles reference stability. "Mostly" is load-bearing: the compiler bails out on components that break the Rules of React, and it bails out on manual memoization that doesn't match what it inferred — which means leftover `useMemo` can cost you the optimization it was meant to provide.

Fiber — when React stopped blocking the browser

Before React 16, the reconciler had a fundamental architectural problem: once it started diffing a component tree, it couldn't stop. It ran synchronously to completion, blocking the main thread the entire time.

On a large tree, this diff could take 50-100ms. During that time, the browser couldn't repaint, handle input events, or run animations. This is why complex React apps felt janky — every state update was a potential freeze.

React Fiber rewrote the reconciler to make work **interruptible**.

The core idea: instead of one big synchronous call stack, React breaks reconciliation into small units — one per component — called fibers. Between each unit, React checks: does the browser need to do something more urgent?

```
Old reconciler: [===== blocks everything for 100ms =====]  
Fiber:          [=]check[=]check[=]check — yields between units
```

If urgent work arrives mid-reconciliation, Fiber pauses, handles the urgent work, and resumes. If new state arrives that makes the current render irrelevant, Fiber throws the in-progress work away and starts fresh with the latest state.

This is called **concurrent rendering** — multiple renders can be in flight simultaneously, but only the latest one commits to the DOM.

Fiber is the foundation everything modern in React is built on. Suspense, streaming, `startTransition` — none of it is possible without an interruptible reconciler. Chapter 6 is entirely about what that makes possible over a network.

`startTransition` — telling React what's urgent

Fiber gave React the ability to prioritise work. `startTransition` is how you expose that priority system to your own code.

```
import { startTransition } from 'react'
```

```
// Without startTransition – both updates block the input
setInputValue(input)
setSearchResults(input) // expensive re-render blocks typing
```

```
// With startTransition – input stays responsive
setInputValue(input)
startTransition(() => {
  // React can interrupt and restart this
  setSearchResults(input)
})
```

Without it, React treats every `setState` as equally urgent. A slow re-render triggered by typing blocks the input from updating — the input feels laggy even though the data is simple.

With it, React prioritises the input update. The expensive search results render happens when the main thread is free, or gets discarded and restarted if the user types again before it finishes.

You won't reach for this often. The signal is specific: an input or interaction feels laggy because a state update is triggering an expensive re-render. That's the exact problem `startTransition` solves, and nothing else.

Hydration mismatch — what actually happens at the reconciler level

Chapter 1 covered hydration from the rendering side. Here's what happens internally.

On the server, React renders your components and produces HTML or an RSC payload. The VDOM is a transient intermediate step — it's used to generate the output and then discarded.

In the browser, React needs to hydrate. It runs your component functions again to reconstruct the expected VDOM in memory. Then it walks the existing server-rendered DOM simultaneously, comparing node by node. If they match, React attaches event listeners in place — no new DOM nodes created, the server’s HTML is reused.

Server: components → VDOM (transient) → HTML → sent to browser

Browser: components → VDOM (fresh reconstruction)

VDOM vs existing DOM → attach listeners (if match)

Your components run twice — once on the server, once in the browser during hydration. They should produce identical output. When they don’t:

In development: React warns loudly, tells you which node mismatched, and tries to continue.

In production: React silently discards the server HTML for that entire subtree and re-renders it from scratch on the client. No error shown. You just lost your SSR benefit for that section — slower paint, potential layout flash as the server HTML gets replaced.

The layout flash is the user-visible symptom. The server painted one thing, React replaces it with something slightly different, there’s a flicker. Most common with timestamps, theme values read from `localStorage`, or anything that differs between server environment and browser environment.

Common causes:

- `new Date()` or `Math.random()` in a component — server and browser run at different times
- `typeof window !== 'undefined'` checks during render — server takes one branch, browser takes another
- Browser extensions mutating the DOM before React hydrates
- Invalid HTML nesting that the browser silently auto-corrects before React sees it

The fix is ensuring server and client render identical output. Browser-only logic belongs in an effect, not in the render function

— and Chapter 10 is about why that’s a definition rather than a workaround.

Signals — a completely different model

Everything we’ve covered so far is React’s world: describe UI as a function of state, produce VDOM, diff, commit. Signals throw that model out entirely.

In React: state changes → re-render component → new VDOM → diff → DOM update

With Signals: state changes → directly update the exact DOM node bound to that value → done

There’s no VDOM, no diffing, no reconciliation. A Signal is a value wrapper that tracks which DOM nodes are subscribed to it. When the value changes, it notifies exactly those nodes — nothing else in the tree is involved.

```
// React – entire component re-runs on every click
function Counter() {
  const [count, setCount] = useState(0)
  return (
    <button onClick={() => setCount(c => c + 1)}>
      {count}
    </button>
  )
}
```

```
// Preact Signals – only the text node updates, runs once
const count = signal(0)
function Counter() {
  return <button onClick={() => count.value++}>{count}</button>
}
```

In the Signals version, clicking the button doesn't re-execute the Counter function. The Signal tells the specific text node in the DOM to update its value. React's entire render-diff-commit cycle is bypassed.

This is called **fine-grained reactivity**. Updates are $O(\text{number of changes})$ rather than $O(\text{size of the component tree})$.

What you lose without the VDOM

Signals aren't strictly better. Removing the VDOM has real costs.

Predictable data flow. React enforces data flows down, events flow up. You can always trace a bug by asking: which state changed, which component owns it, what did it render. With Signals, state lives outside the component tree and can be read or written from anywhere. That's powerful, and a debugging nightmare on a large team. React's "annoying" unidirectional data flow is a forcing function for maintainable architecture.

Concurrent rendering. React Fiber's scheduling model — `startTransition`, interruptible rendering, priority lanes — is built on the VDOM as an intermediate representation. When React pauses mid-render, it's pausing VDOM construction. The real DOM is never in a half-finished state. Signals update the DOM synchronously and directly. You can't deprioritise a Signal update the way you can with `startTransition`.

The ecosystem. Error boundaries, Suspense, RSC, DevTools — all built on the VDOM as a shared representation of UI. Signals-based frameworks have their own tooling, but it's a fraction of React's maturity.

Why React hasn't adopted Signals

Every major framework except React has moved toward Signals or fine-grained reactivity: Vue's `ref()`, Solid's native signals, Svelte 5's runes, Angular's signals. React is the holdout.

The React team's official position: React Compiler makes Signals unnecessary. If the compiler automatically memoizes everything at build time, the wasted re-render problem disappears without changing the programming model or the data flow guarantees.

The counterargument: the compiler is a build-time heuristic. It analyses your code statically and can bail out on complex patterns. Signals are a runtime guarantee — a Signal update always touches exactly the nodes bound to it, no analysis required.

Both positions are defensible. React's bet is that compiler technology catches up and developers keep the familiar model. The Signals camp's bet is that fine-grained reactivity is the correct primitive and no compiler fully replicates it.

What's clear is that frameworks starting fresh have consistently chosen fine-grained reactivity, while React is defending the VDOM model with a compiler rather than with the model itself. Whether that's React being late or React being right is not settled, and anyone telling you otherwise is selling something.

This trade — one coherent pass you can reason about, versus precise subscription — shows up again in Chapter 9, at a completely different layer of the stack. It's worth noticing that the industry keeps making it in the same direction.

The full picture

React's architecture is a set of deliberate tradeoffs, not a collection of optimal solutions.

Mechanism	What it buys	What it costs
VDOM	A cheap scratchpad for UI descriptions	JS re-execution on every render
Reconciliation	Minimal DOM updates from the diff	Only O(n) because of the type and key assumptions
Keys	Stable identity across list renders	Index keys silently break on mutation
Fiber	Interruptible, prioritised rendering	Complexity, concurrent-mode edge cases
memo	Prevents child re-renders	Fails silently on reference equality
useMemo / useCallback	Stabilise references so memo works	Boilerplate, easy to get wrong
React Compiler	Automatic memoization at build time	A black box that bails out on complex patterns
Signals	Direct DOM updates, no diffing	Loses unidirectional flow and concurrent mode

None of these are mistakes. Each one is a choice about what to optimise for — developer experience, correctness guarantees, runtime performance, or architectural predictability.

Understanding the tradeoffs is what separates reaching for memo out of habit from knowing exactly why you’re reaching for it — and whether you actually need to.

That’s React’s engine. The next chapter asks a different question: not *how* it updates the DOM, but *when any of it runs at all*, and what comes out of each step.

Chapter 3 — What Runs When, and What Comes Out

Here's a test. Take this component:

```
async function PostList() {
  const posts = await db.posts.findMany()
  return (
    <ul>{posts.map(p => <li key={p.id}>{p.title}</li>)}</ul>
  )
}
```

Answer four questions without looking anything up:

1. Does that database query run when you deploy, or when a user visits?
2. Does it produce HTML, or something else?
3. If a user is already on your site and clicks a link to this page, does it produce the same thing?
4. How many times does the `<li>` get created?

Be honest. Or you won't learn.

I could name every piece of this system and still not answer question 3 with confidence. Knowing the parts isn't the same as knowing the schedule. This chapter is the schedule.

First, what “render” even means

Three different things, and this book is about to use all of them.

React's meaning: call a function, get an object.

```
function Post() { return <h1>Hello</h1> }
```

Calling that gives you roughly `{ type: 'h1', props: { children: 'Hello' } }`. A plain JavaScript object. That's a React render, and it needs no browser, no screen, and no DOM.

The browser's meaning: turn markup into pixels. Parse HTML into a DOM tree, parse CSS into a CSSOM, compute styles, run layout to decide where every box sits, paint, composite. This is the browser engine — Blink, WebKit, Gecko. It's the meaning you're probably picturing.

"Server-side rendering," which is neither of those. It means: produce an HTML string.

The server does the first and the third. **It never does the second.**

So the server never touches your CSS

It calls your functions, gets objects back, walks the tree and concatenates strings. The API is literally called `renderToString`. `<div className="card">` becomes the characters `<div class="card">` — and nothing anywhere asked what `.card` looks like, because nothing on the server could answer.

No CSSOM. No cascade. No layout. No box model.

Which explains a set of errors you've probably hit without connecting them:

- **`getBoundingClientRect()` doesn't work on the server.** There's no layout, so nothing has a size or a position. Measuring anything requires the browser to have finished, which is why measurement always ends up in an effect.
- **Media queries can't be evaluated.** There's no viewport to measure. A component that renders differently at mobile width has to guess on the server and correct in the browser — which is a hydration mismatch by construction.
- **`window` and `document` don't exist**, for the same reason. They're browser-engine objects and there's no engine.

Why the word is confusing

React took “render” from graphics, where it does mean producing pixels, and kept it for “produce a description.” We’ve been paying for that ever since.

The proof that React’s render has nothing to do with HTML: React Native runs the same render and produces native views. Same components, same reconciliation, no DOM anywhere.

React’s core reconciles a tree of descriptions. A **renderer** decides what to do with them:

- react-dom/client creates and mutates DOM nodes
- react-dom/server builds an HTML string
- react-server-dom-* writes a payload
- react-native creates native views

Keep that split in mind for the rest of this chapter. When something below “renders on the server,” it means functions ran and text came out. Nothing was drawn.

Three artifacts

Everything the system produces is one of three things. Get these straight and the rest follows.

HTML. Text the browser can paint without running any of your JavaScript. Final. Once it exists, the information about which component made it is gone.

The RSC payload. A text description of a component tree — what elements exist, what props they have, and where the client components go. It’s *data about a tree*, not a rendered tree. React calls the format Flight. It looks like this:

```
1:I[["./components/LikeButton.js"],["chunks/main.js"],"default"]
2:["$", "article", null, {"children": "$1"}]
```

You will never write one and you will never parse one — you just need to recognise it in the Network tab when it shows up as `text/x-component`.

DOM mutations. What React does in the browser once it's running.

Two relationships between them, and you need both.

First: the payload comes first, and HTML is generated from it. Not the other way around, not in parallel. Server Components never produce HTML directly. They produce the payload, and a second renderer turns the payload into HTML.

Second: HTML is a lossy copy. Turning a payload into HTML throws information away permanently. `<article><ul><li>First post</li></ul></article>` doesn't record which component produced the `<article>`, where the boundaries between components are, or which props anything received. The payload has all of that; the HTML has none of it.

Those two facts together explain something that otherwise looks redundant. When you load a page fresh, you receive **both** artifacts. That seems like waste — if HTML is made from the payload, why send the source as well?

Because they do different jobs:

- **HTML** can be painted without running any JavaScript. It's what you see before React exists.
- **The payload** is the only copy that still describes the tree. React needs it to know what the tree *is*, so it can reconcile against it later.

Ship only HTML and React has a picture of a tree with no idea what tree it's a picture of. It couldn't hydrate, and it couldn't diff anything against it on the first navigation. HTML is what the page looks like. The payload is what the page *is*.

The same file runs at different times

This is the thing nobody says plainly.

`PostList.tsx` is not “a build-time component” or “a request-time component.” It’s a function. It runs whenever the framework decides to run it, and that decision is made per route, not per file. The exact same code can execute at build time on Monday and on every request on Tuesday, because you changed one line somewhere else.

So “when does this run” is never a property of the component. Stop looking for it there.

Then what decides? In Next.js 16, the default is request time — every route runs fresh unless you explicitly cache it with `use cache`. That’s the whole answer for now. The details of *how* caching moves work from request time to build time are Chapter 11, deliberately, because you can’t reason about the optimization until you can reason about the thing being optimized.

Moment 1: next build

For a route Next decides to prerender:

First, `generateStaticParams` runs — if the route has dynamic segments. `/[[locale]]/property/[[id]]` can’t be prerendered until something says which locales and which ids exist. This function returns that list, and it decides how many times everything below happens. No dynamic segments, no list needed, and this step doesn’t exist.

When you run a build on Next 16, this is what you can expect:

```
f Proxy (Middleware)
```

- o (Static) prerendered as static content
- (SSG) prerendered as static HTML (enumerated paths)
- f (Dynamic) server-rendered on demand

This is the whole difference between o (Static) and ● (SSG) in your build output.

Both are prerendered at build time, both write files to disk, both execute nothing on the server at request time. ● just means the route had holes in its path and you enumerated them, so it produced N outputs instead of one. Same mechanism, different arity. The “SSG” label is a leftover from the `getStaticProps` era.

Two things that trip people up here. The symbol describes the **full route path**, not the page you’re looking at — `/login` in an app with a root `[locale]` segment is really `/[locale]/login`, so it’s ● no matter how static that page feels. In an app with a dynamic root segment, nothing is ever o.

And you may well have never written `generateStaticParams` while seeing ● all over your build output. It usually arrives as boilerplate: i18n libraries put it in `app/[locale]/layout.tsx` during setup and you never look at it again.

Your Server Components execute. In Node, on your machine or your CI runner. `await db.posts.findMany()` fires now. Whatever the database says at 2pm on deploy day is what gets baked in. Once per param combination, if there were params.

They produce a payload. Not HTML. The payload.

And where a client component was, the payload has a hole with an address on it.

This is the part that’s easy to miss. When the server render hits `<LikeButton />` and sees “use client”, it does not execute the component. It writes down where to find it:

```
1:I["./components/LikeButton.js",["chunks/main.js"],"default"]
```

A module path, the chunk that contains it, and an export name. That’s the whole entry. No code, no output, no props resolved — an address, and a slot in the tree where whatever lives at that address belongs.

Something else has to turn that address into a real component, and that something is a **manifest** — a lookup table the bundler wrote at build time, mapping module IDs to the chunks that hold them. For now the only thing that matters is that the payload is full of addresses, and resolving them is a separate job from producing them.

Then a second renderer runs. It reads that payload, and every time it hits one of those addresses, it looks the module up in the server manifest, loads the real component, and executes its render function and appends the resulting HTML string.

Read that again, because it's the most counterintuitive fact in this entire model:

Your Client Components run on the server.

"use client" does not mean "client only." It means **also client**. The directive marks a boundary, not a location — and everything past that boundary still renders on the server first.

Don't confuse it with "state and effect" running on the server.

You have already seen the proof of this, probably many times:

- **ReferenceError: window is not defined** in a "use client" component. That's Node telling you it just tried to execute your component.
- **Hydration mismatch errors.** A mismatch means two renders disagreed. If client components only ran in the browser, there'd be nothing for the browser to disagree *with*.
- **Pages Router.** Every component in a Next.js app has always rendered on the server. That didn't change with App Router. What changed is that some components got a directive and an unfortunate name.

Check it yourself in thirty seconds. Open a "use client" page in a fresh tab — type the URL, don't click a link to it — and hit Ctrl+U for View Source. Search for any text your client component renders.

It's there. In the raw HTML document, before a single line of your JavaScript has run.

Use View Source, not the Elements panel. Elements shows you the live DOM after hydration, which proves nothing — of course the component rendered, you’re looking at the result. View Source shows the bytes the server actually sent.

And don’t use a `console.log` for this. It’s an unreliable test: Fast Refresh re-runs your component in the browser only, so a log you just added won’t appear in the terminal until the next full request. And if you reached the page by clicking a link, there was no SSR pass at all — see Moment 4. Both produce silence in your terminal for reasons that have nothing to do with whether the claim is true.

The exception: a component imported through `next/dynamic` with `ssr: false` genuinely doesn’t render on the server. Next emits a placeholder and the component first executes in the browser. That’s opt-out behaviour, and it’s the only way to get it.

What actually runs in that pass

Only the render function body. Specifically:

- `useState` returns its initial value and nothing ever updates it
- `useEffect` and `useLayoutEffect` do not run at all
- Refs are not attached
- Event handlers are not attached — `onClick` is a function, and functions can’t be written into HTML

One pass, to produce markup. Then that component is finished until the browser picks it up.

The reason for the whole exercise is the obvious one: without it, every client component would be a blank hole in your HTML until JavaScript finished downloading. The SSR pass is what makes the page look complete before it becomes interactive — which is exactly the hydration gap Chapter 1 described.

Two files get written to disk. An `.html` file and an `.rsc` file. Same route, same moment, two artifacts. Remember that both exist; it matters in a minute.

Moment 2: a request comes in

Two cases, and the difference is which artifacts already exist.

If the route was prerendered: Next reads the `.html` and `.rsc` files off disk and sends them. **On the server, nothing executes** — no component runs, no query fires, no rendering happens. That’s the entire point of prerendering.¹

Say that precisely, because the imprecise version causes a real misunderstanding. Prerendering removes the *server’s* request-time work. It removes nothing from the browser. Hydration still happens, your Client Components still run, and everything in Chapter 2 about reconciliation still applies in full. A prerendered page and a dynamic page are identical from the moment the response lands.

If it wasn’t: everything from Moment 1 happens now instead, per request, streaming.

- Server Components execute, in Node, on your server
- They produce a payload
- The SSR pass consumes it and produces HTML
- Both stream to the browser in one response

Which brings us to the thing that confuses everyone.

Why the first response contains both

Open View Source on a Next.js page. Scroll to the bottom. You’ll find a pile of `<script>` tags pushing strings into an array.

That’s the payload, inlined into the HTML document.

¹One asterisk, covered in the appendix to this chapter: `proxy.ts` runs on every request regardless, before any of this. It’s the single exception to “nothing executes.”

The browser gets both artifacts in the same response, and it needs both, for different reasons:

- **The HTML** so it can paint immediately, before any JavaScript has downloaded
- **The payload** so that when React does start up, it knows what the tree is supposed to be

Without the HTML you'd stare at a blank screen. Without the payload React would have to guess what tree the server had in mind, and it can't — HTML has thrown that information away.

I know this is a little repetitive, we covered it already, but it's important to get comfortable with the idea.

Moment 3: the browser wakes up

The HTML has painted. Nothing is interactive yet. Then:

JavaScript chunks download. Only client component code. Your Server Components' code was compiled into the server bundle under `.next/server/`, and it stays there — it isn't in anything the browser downloads, and it never will be.

React reads the inlined payload and reconstructs the tree in memory — holes, addresses and all.

The same addresses get resolved a second time, differently.

```
1:I["./components/LikeButton.js",["chunks/main.js"],"default"]
```

That meant “load this out of `.next/server/`” on the server. In the browser it means “make sure `chunks/main.js` has been downloaded, then take its default export.” Same address, same slot in the tree, two completely different lookups against two different manifests.

That's also how the browser knows which downloaded chunk belongs to which piece of the page. It isn't inferring it from the DOM — the payload told it, by address.

Client components execute again. Second time for these — once on the server, once here. React walks the existing DOM, matches it against what these render functions produce, and attaches event handlers.

That’s hydration. React isn’t creating DOM here; it’s adopting DOM that’s already on the page. When you get a hydration mismatch error, this is the moment it happens — the browser run produced something different from the server run.

Now onClick works.

The count

For one component on one page load, here’s how many times each one actually runs.

	Server Component	Client Component
RSC render (server)	Runs	Doesn’t run — an address is emitted instead
SSR pass (server)	Doesn’t run — it already produced its output	Runs , producing markup
Hydration (browser)	Never — its code isn’t there	Runs again , attaching handlers
Total before interactive	Once. Ever.	Twice

And then as many more times after that as state changes require.

But React builds a tree of everything. Doesn’t it need the code?

Reasonable objection, especially straight after Chapter 2. If React reconciles the whole tree in the browser, and Server Components are part of that tree, how does it manage without them?

The distinction is between **elements** and **the code that produced them**.

When `PostList` runs on the server, it returns elements — a `<ul>`, some `<li>`s. Those elements go into the payload. The browser reads the payload and puts them into its tree. Everything from Chapter 2 applies to them without exception: they’re reconciled, diffed, keyed, committed. React does not treat them as second-class.

What the browser doesn’t have is the `PostList` *function*.

So the browser can do one of those things and not the other:

- It **can** diff those elements against a newer version of them
- It **cannot** produce a newer version, because it has no way to invoke the function

That single asymmetry explains most of the model. It’s why a link click has to go back to the server — not because the browser is missing elements, but because the browser cannot compute new ones and a fresh payload is the only way to get them. And it’s a better explanation of “Server Components can’t have state” than the usual one: state means something re-runs when a value changes, and nothing in the browser is capable of re-running these.

The browser holds the output. The server holds the function. That’s the deal.

Moment 4: the user clicks a link

Here’s question 3, and here’s where the model usually breaks.

The user is on `/posts` and clicks a `<Link>` to `/posts/42`. What happens?

On the server: the Server Components for `/posts/42` execute. Database query fires. A payload is produced.

And then it stops. No SSR pass. No HTML. None is generated, because none is needed — there’s already a live React app in the browser, and it doesn’t want a document. It wants a tree description.

In the browser: React takes the new payload and reconciles it against the tree it's already got. Layouts that didn't change don't re-render. Client component state in the parts of the tree that survived is preserved. New client components in the new route execute for the first time.

So: **same route, same components, different response depending on how you arrived.** Type the URL and hit enter, and the server sends a document — HTML plus the payload inlined in it, whether those were built just now or read off disk. Click a link to that exact same page, and the server sends a payload and nothing else.

That asymmetry is the answer to question 3, and it's why “server-rendered” is a misleading description of a Next.js app. Most page views in a real session never produce a single byte of HTML.

Check it yourself. Open the Network tab, click a `<Link>`, and look for the request with `?_rsc=` in the query string. Content type `text/x-component`. There is no document request. That's the whole thing, visible in five seconds.

Moment 5: interaction

The user clicks a button. State changes.

Only client component code runs. Only in the browser. No server involvement, no payload, no HTML. React re-renders the affected subtree and mutates the DOM.

This is ordinary React, and it's the *only* moment that is.

Moment 6: a mutation

The user submits a form backed by a Server Action.

A POST goes to your own server. Not a route — the action itself, addressed by an ID the bundler generated.

The function executes in Node. Database write happens.

The response is a payload. The action’s return value, and — if you revalidated — a freshly rendered tree for the affected route.

Still no HTML. React reconciles the new tree in.

For now the only thing to hold onto: **mutations return trees, not JSON.**

The whole schedule

Seven moments. For each one: what runs, where it runs, and what comes out.

Moment	What runs	Where	What comes out
next build	Server Components, then Client Components in the SSR pass	Node	HTML and a payload, written to disk
First request, prerendered route	Nothing	—	The two files are read off disk and sent
First request, dynamic route	Server Components, then Client Components in the SSR pass	Node	HTML and a payload, streamed

Moment	What runs	Where	What comes out
Hydration	Client Components	Browser	Event handlers attached to DOM that's already on the page
Link click	Server Components only	Node	A payload. No HTML
Interaction	Client Components only	Browser	DOM mutations. Nothing leaves the machine
Server Action	The action itself, then Server Components if you revalidated	Node	A payload. Still no HTML

Read the “what comes out” column on its own and the shape of the whole system shows up: **HTML is generated exactly twice — at build, or on a document request. Every other moment produces a payload or nothing at all.**

Back to the four questions

1. **Build or request?** Neither, inherently. Request time by default in Next 16, build time if you cache it. It’s a property of the route’s configuration, not of the component.

2. **HTML or something else?** A payload, always and first. HTML is generated *from* the payload, and only when a browser needs a document.
 3. **Same thing on a link click?** No. Payload only. No HTML is produced at all.
 4. **How many times does the `<li>` get created?** Once as a description on the server. Once as HTML text, if this was a document request. Once as a real DOM node in the browser. Three representations of one `<li>`, and only the last one exists in the page.
-

Appendix: the one thing that always runs

Moment 2 said a prerendered route executes nothing on the server. That's true of *your route*, and there's exactly one exception sitting in front of it.

proxy.ts runs on every request. Always. Before routing, before rendering, before caching helps you. Prerendered or dynamic, first load or navigation — if a request reaches your app, this file ran first. It's the one piece of your code with no "it depends."

Through Next 15 it was called `middleware.ts`. Next 16 renamed it to `proxy.ts` (there's a codemod), and the rename is a correction, not cosmetics. "Middleware" is an Express word, and in Express middleware is where you *do things* — parsing, auth, logging, business logic. That instinct is harmful here, because this file sits at your app's network boundary, on the path of every request. Work you do here is work every request pays for, before anything else runs. "Proxy" is the honest name: inspect a request and decide what happens to it. Direct traffic, don't do work.

What it's for is four decisions about a request, never work on it: **rewrites** (show one URL, serve another), **redirects** (send it elsewhere before it hits a route), **headers** (read a cookie, set a security header), and **gating** (decide whether it proceeds at all).

```
// proxy.ts
import { NextResponse } from 'next/server'
import type { NextRequest } from 'next/server'

export function proxy(request: NextRequest) {
  const token = request.cookies.get('session')
  const { pathname } = request.nextUrl
  const isDashboard = pathname.startsWith('/dashboard')
  if (!token && isDashboard) {
    return NextResponse.redirect(new URL('/login', request.url))
  }
  return NextResponse.next()
}
```

`NextResponse.next()` means proceed; `redirect` or `rewrite` means don't, do this instead. That's the whole vocabulary.

Three things worth knowing.

It runs on Node. `proxy.ts` runs on the Node.js runtime. That's a change from the old middleware, which defaulted to Edge and its restricted API surface — the reason you used to reach for `jose` instead of `jsonwebtoken` in a middleware file. In 16 that constraint is gone. If you specifically need Edge behaviour, that's the one case to stay on `middleware.ts` for now.

The matcher is a security surface. Left alone, `proxy.ts` runs on everything — every page, every asset, your `favicon.ico`. `config.matcher` narrows it. Too broad wastes cycles on the busiest code path you have; too narrow silently skips the check on the one route that needed it. Neither fails loudly.

```
export const config = {
  matcher: ['/dashboard/:path*', '/account/:path*'],
}
```

Gating here is not authorization. A proxy redirect is a routing decision — a good cheap first gate, not the thing between

an attacker and your data. Every Server Action is a public endpoint: the framework generates an ID for it, that ID ships to the browser, and anyone can POST to it directly. Chapter 7 takes that apart properly. What matters here is the consequence — a proxy that guards *pages* does nothing for the *actions* behind them, because those are never routed through a page request at all. So the real check lives on the data access, in the Server Component or the Action, every time. If you centralised all your auth in `proxy.ts` because it felt like the one clean place, you built a single point of failure guarding the front door while the side doors stayed open. The rename is partly a nudge toward doing less here, not more.

Chapter 4 — What Your Bundler Actually Does

Browsers have supported ES modules natively since 2018. Every browser you care about can read `import` and `export` without help.

So why is there still a build step that takes your clean module graph and produces `00d.e4.nsryo4.js`?

First, the graph

Everything in this chapter operates on a structure called the module graph, so it's worth four paragraphs on where that structure comes from and why it exists.

import statements are static. You cannot write `if (condition) { import x from './y' }` — it's a syntax error. Imports are resolved at parse time, before any of your code executes. That restriction is the whole point: because the import statements can be read without running anything, a tool can build a complete picture of which files reach which other files, ahead of time. Nodes are modules, edges are imports. That's the graph.

Tree shaking is what the graph buys you. Once a tool knows the full set of edges, it can ask which modules and which exported bindings nothing ever reaches, and drop them from the output. This is dead code elimination, and it works reliably with ES modules for exactly the reason above. CommonJS `require()` is a function call evaluated at runtime, so the same analysis is guesswork — which is why CJS packages are harder to shake.

export * is where it goes wrong. A barrel file — `components/index.ts` re-exporting forty modules — creates an edge from the barrel to every one of them. Importing one button from the barrel makes

the barrel reachable, and depending on how much the bundler can prove is side-effect-free, some or all of those forty may come along. The fix is to import from the source module directly.

sideEffects is how a package says it's safe to drop. A module that merely defines and exports things can be removed when unused. A module that does something on import — registers a polyfill, injects CSS — cannot. Bundlers can't always tell the difference, so `package.json` carries the answer:

```
{ "sideEffects": false }
```

Or per-file, when only some modules are safe:

```
{ "sideEffects": ["/src/polyfills.js", "*.css"] }
```

That's the graph, and what can be removed from it. This chapter is about what happens to it afterwards, when something has to turn a few hundred module records into a handful of files a browser can actually fetch.

Why bundling survived native ESM

The obvious answer is “fewer requests,” and it's the wrong one. HTTP/2 multiplexes; a hundred small requests aren't a hundred round trips.

The real problem is **depth**.

Your app imports a component, which imports a hook, which imports a utility, which imports a date library. The browser cannot discover that date library until it has downloaded and parsed the three files above it. Each layer costs a full round trip, and the browser can't parallelise what it hasn't learned about yet.

A deep graph on native ESM turns into a staircase. On a fast connection you might not notice. On hotel wifi at 200ms latency,

a five-level import chain is a second of nothing before anything renders.

A bundler knows the whole graph at build time. It can put the date library in the same file as the component that eventually needs it, and now that staircase is one request.

Two smaller reasons, both real:

- **Bare specifiers.** `import _ from 'lodash'` means nothing to a browser. There's no `node_modules` on the web. Something has to turn that into a URL.
 - **Your source isn't JavaScript.** TypeScript and JSX are not things a browser can run. Something has to compile them regardless of bundling.
-

The pipeline

Every bundler, including Turbopack, does the same sequence. The differences are in speed and strategy, not in shape.

Resolve. Turn `import { formatDate } from './utils'` into an absolute file path. This is more work than it sounds — extension guessing, `node_modules` directory walking, `package.json` exports maps, and **conditions**.

Conditions are worth pausing on. A package can ship different files for different environments, and the bundler picks based on which condition names are active — `import` versus `require`, `browser` versus `node`, and one you'll meet properly in Chapter 5: `react-server`. The same `import` statement in the same file can resolve to two different files depending on which graph is being built. Hold onto that.

Load. Read the bytes off disk.

Transform. TypeScript strips to JavaScript, JSX compiles to function calls. Next uses SWC for this — a Rust compiler doing what Babel used to do, faster. Nothing clever here, just translation.

Parse. Turn source text into an AST, and read the `import` and `export` statements out of it. This is the step that only works because ESM is statically analysable.

Build the graph. Take every import you found, resolve it, load it, transform it, parse it, repeat. Stop when you run out of new modules. Now you have every file your app can reach, and the edges between them.

Everything up to here is bookkeeping. The interesting part is next.

Chunking: how the bundler decides what goes in which file

You have a graph of, say, 1,400 modules. You need to emit files. How many, and which modules go in which one?

This is the part most developers have never thought about, and it's the part with real consequences.

The two failure modes bound the problem:

- **One giant file.** Every route downloads every byte of your app. Compression is excellent. Change one line and the whole thing invalidates for every returning user.
- **One file per module.** Perfect caching granularity, perfect precision. Fourteen hundred requests, terrible compression, and you've reinvented the staircase you were trying to avoid.

Every real strategy lives between those.

Entry points

Chunking starts from entries — the places a browser begins loading. In an App Router app that's roughly one per route, plus shared framework code.

From each entry the bundler walks the graph and collects everything reachable. Two routes will overlap heavily; both need

React, both need your design system. Emitting that shared code twice is waste, so it gets pulled into a chunk both routes load.

import() is a split point

This is the one lever you control directly.

```
const Chart = lazy(() => import('./HeavyChart'))
```

That isn't a hint. The bundler treats a dynamic import as a hard boundary — `HeavyChart` and everything only it reaches go into a separate file, not loaded until that `import()` executes.

Every static `import` says “this must be present before my module runs.” Every dynamic `import()` says “this can arrive later.” That's the entire API for controlling your bundle, and it's two characters.

Chunk groups

Turbopack's model is worth knowing because it's a genuinely clean idea.

A **chunk group** is the set of chunks that get loaded together. Everything `/home` needs is one group; everything `/blog` needs is another. A shared chunk can belong to both.

Turbopack starts by producing many small chunks — far more than webpack did — and then merges them back up. The rule for merging is the interesting bit: **it only merges chunks that belong to the same group.**

The reason is the over-shipping problem. If you merge a `/home-only` chunk into a `/blog-only` chunk, every visitor to `/blog` now downloads code that only `/home` uses. But merging two chunks *within* the `/home` group costs nothing, because anyone loading one was already loading the other. The merge saves a request and adds no bytes.

That constraint is what lets it be aggressive about reducing file count without accidentally shipping your admin dashboard to your marketing page.

The thresholds are configurable, and the defaults tell you how Turbopack thinks:

```
// next.config.ts
const nextConfig = {
  experimental: {
    turbopackChunking: {
      minChunkSize: 50000,
      maxChunkCountPerGroup: 40,
      maxMergeChunkSize: 200000,
    },
  },
}
```

Read those as: don't bother making chunks smaller than 50KB, don't let a route need more than 40 files, don't merge past 200KB.

One catch worth internalising — **those numbers are uncompressed, unminified bytes**. Roughly five times the size of what actually ships. A 50KB `minChunkSize` is about 10KB on the wire. If you go tuning these, you'll be off by 5x in your head otherwise.

Module IDs, and why they decide your cache hit rate

Inside a chunk, your modules don't have file paths anymore. Each one gets an ID, and imports between them become lookups in a registry.

That sounds like an implementation detail. It's a caching mechanism.

Chunk filenames contain a content hash. Change the content, change the hash, change the URL, and every returning user downloads it again. So if module IDs were assigned in, say, the

order the bundler happened to discover them, then adding one import to one file could shift the IDs of hundreds of modules — changing content in chunks you never touched, invalidating caches across your whole app for a one-line change.

So IDs are assigned deterministically. Same input, same ID, regardless of build order or machine. Your chunk hashes only move when the code in them actually moves.

And one of those IDs is the address from Chapter 3.

```
1:I["./components/LikeButton.js",["chunks/main.js"],"default"]
```

That module path and that chunk name are bundler output. The RSC payload can only point at a client component because the bundler assigned it a stable identity and wrote down which file it lives in. Chapter 5 is about the table that gets written down.

Minification is not tree shaking

Commonly conflated, entirely different passes.

Tree shaking works on the graph, before chunking. It asks which modules and exports nothing reaches, and drops them — `sideEffects, /##_PURE_*/`, the export * problem from the opening of this chapter.

Minification works on syntax, after chunking. It renames local variables to single letters, deletes whitespace and comments, folds constants, collapses `if (true)`. It has no idea what a module is. It's a text transformation that preserves behaviour.

Both make the bundle smaller and that's their only similarity. Tree shaking removes code you never reached; minification shrinks code you did.

Source maps are the third piece here. Once names are mangled and files are concatenated, a production stack trace points at line 1, column 48,203 of a file that doesn't exist in your repo. A source map is the lookup table back — which is why an error monitoring service that can't find your source maps is nearly useless.

Your dev bundle is not your prod bundle

They're built differently, on purpose, and this is where people mislead themselves.

Dev optimises for the edit loop. Turbopack compiles lazily — code for routes you haven't visited isn't compiled at all. Module boundaries stay intact so Fast Refresh can swap one module without reloading the page. Nothing is minified, everything is source-mapped, and chunking barely happens.

Prod optimises for delivery. Full graph, real chunking, minification, content hashes.

Which means dev tells you nothing about bundle size. If you want to know what you're shipping, you have to build. This is also why a component can work perfectly in dev and break in prod: the dev bundle never exercised the chunking or the minifier.

What changed in Next 16

Turbopack is now the default for both `next dev` and `next build`. Two consequences you'll actually see.

More files. Webpack packed aggressively — a handful of large chunks like `main-abc123.js` and `commons-def456.js`. Turbopack produces many small, granular chunks before merging. Look in `.next/static/chunks` after a build and expect dozens where you used to see a few.

That's better caching, worse deploy ergonomics. If you sync build output to object storage, you now have far more files, and content-hashed names mean every build creates new ones. Old chunks accumulate unless you're deleting what's no longer in the output.

One graph instead of three. Webpack ran separate compiler processes for client, server and edge, then stitched their outputs

together. Turbopack keeps a single logical graph that understands multiple environments — the same module resolves differently depending on which environment is pulling it in.

That's not a performance footnote. It's the machinery that makes the next chapter possible, and it's why "the same file exists in two graphs at once" is a coherent statement rather than a contradiction.

Go and look at your own build

Twenty minutes, and it's more useful than the rest of this chapter.

```
next build
ls .next/static/chunks
```

Count them. Look at the sizes. Find the biggest one.

Then install `@next/bundle-analyzer` and open the treemap. You're looking for exactly one thing: **something large that you don't recognise**. A date library you imported once for one format call. An icon set where you used four icons. A component that pulled its entire package in through a barrel file.

Everything above applies to any JavaScript app. Nothing in it is React-specific.

But Chapter 3 left a loose end: the payload contains addresses, and something resolves them — a manifest, built at build time, mapping module IDs to chunks. That manifest is written by the bundler, which means React's server-side render depends on bundler output to function at all.

That's an unusual dependency, and it has consequences: two module graphs from one repo, imports that fail in one direction and silently poison in the other, and a single misplaced "use client" that can put your whole app in the browser's download.

References

- Next.js — How Turbopack chunks your JavaScript
- Next.js — `turbopackChunking` configuration reference
- Dan Abramov — *Why Does RSC Integrate with a Bundler?*

Chapter 5 — Two Graphs, One Repo

Chapter 4 ended on a loose thread. The payload from Chapter 3 contains an address:

```
1:I["./components/LikeButton.js",["chunks/main.js"],"default"]
```

A module path, a chunk, an export name. That's bundler output — the bundler assigned the ID and decided which file the module lives in.

Which means React's server renderer cannot produce a payload without knowing what the bundler did.

Sit with that for a second, because it's genuinely strange. React is a UI library. Bundlers are build tooling. For twenty years the relationship was one-directional: you wrote React, a bundler packaged it, and React neither knew nor cared what happened afterwards.

RSC breaks that. The server render depends on build output to function at all.

Most people accept "React needs a bundler now" as trivia rather than noticing it's a genuinely unusual coupling. It's worth noticing, because everything awkward in this chapter follows from it.

What "use client" compiles to

Chapter 3 said the directive marks a boundary rather than a location. Here's the mechanical version.

When the bundler builds the **server** graph and hits a module with "use client" at the top, it doesn't compile that module into

the server bundle. It substitutes a stand-in — an object carrying the module's ID, its chunk, and the export name. Every import of `LikeButton` in server code resolves to that stand-in.

So on the server, `LikeButton` is not a function. It's a small object that says *here is where the real thing lives*. When the renderer encounters it in the tree, it can't call it — there's nothing to call — so it writes the address into the payload and moves on.

That's the whole trick. The directive is an instruction to the bundler to *not include this code here, and leave a forwarding note instead*.

And it explains a detail from Chapter 3 that otherwise looks arbitrary. Props passed to a client component have to be serializable — not because React is being fussy, but because those props are going into a text payload alongside an address. There's no function on the other side to receive a closure.

Two graphs

If the server gets a stand-in and the browser gets the real module, then the same file is being treated two different ways in one build. That's not a metaphor. There are two module graphs.

The server graph starts at your routes and layouts. It contains every Server Component, every server-side utility, your database client, your secrets. "use client" modules appear in it only as stand-ins.

The client graph starts at every "use client" module. Those are its entry points. From each one it follows imports outward, and everything reachable ends up in a chunk the browser downloads.

One repo. One src directory. Two graphs built from it, with different entry points and different rules.

Turbopack, the default since Next 16, doesn't literally run two separate compilers the way webpack did — it keeps a single logical graph that understands multiple output environments, and resolves

a module differently depending on which environment is pulling it in. Same outcome, less duplicated work.

The mechanism: export conditions

Chapter 4 mentioned conditions during resolution — `import` versus `require`, browser versus node. There's another one, and it's what makes two graphs possible: **react-server**.

A package can ship different files for different conditions:

```
{
  "exports": {
    ".": {
      "react-server": "./dist/server.js",
      "default": "./dist/client.js"
    }
  }
}
```

The `react-server` condition is active while building the server graph and inactive while building the client graph. So `import { thing } from 'some-package'` in one file can resolve to two entirely different files depending on which graph is asking.

That isn't an edge case. It's how React itself ships — `react` resolves to a different build under the `react-server` condition, one where `useState` doesn't exist.

Which gives a much better answer to a rule you've probably memorised. Server Components can't use hooks, and the usual explanation is that they don't support them. The real explanation is that **in that graph, the hooks aren't there**. Nothing is refusing. There's simply no `useState` to import.

The manifest

Two graphs need a phone book between them. That's the client manifest — a JSON file the bundler writes at build time, mapping module IDs to the chunks that contain them and the exports they provide.

It gets read twice, by two different consumers, exactly as Chapter 3 described:

- **The SSR pass**, in Node, resolving an address to a module in `.next/server/` so it can render markup
- **The browser**, at hydration, resolving the same address to a downloaded chunk

Same address, two manifests, two lookups.

This is the concrete answer to “why does RSC need a bundler.” Not for performance. Not for convenience. The protocol *contains bundler identifiers*, and nothing can consume it without the table that decodes them.

Imports across the boundary

Two graphs, so imports can cross in two directions. They behave completely differently, and only one of them is safe.

Server importing client: fine

```
// app/page.tsx – server
import { LikeButton } from './LikeButton' // "use client"
```

This is the normal case. The bundler swaps in a stand-in, the payload gets an address, everything works.

Client importing server: this is where it goes wrong

```
// components/LikeButton.tsx - "use client"  
import { getUser } from '../lib/db' // no directive
```

`db.ts` has no directive. So the bundler, walking outward from a client entry point, treats it as client code — and pulls it into the client graph.

Your database module is now in a browser chunk.

Sometimes this fails loudly: `db.ts` imports `fs`, there's no `fs` in a browser, the build breaks and you go fix it. That's the good outcome.

Sometimes it doesn't fail at all. The module is pure JavaScript, it bundles cleanly, and if it read `process.env.DATABASE_URL` at module scope, that value is now a string literal in a file anyone can download.

A file with no directive isn't "server code." It's code that belongs to whichever graph imports it. Server Components are unambiguous because they're only reachable from the server entry points. A utility module is whatever pulls it in.

server-only, and how it actually works

The fix is a package that makes the mistake loud:

```
import 'server-only'
```

The implementation is a neat trick and worth knowing because it teaches you conditions properly. The package ships two files. Under the `react-server` condition it resolves to an empty module that does nothing. Under any other condition it resolves to a module that throws.

So in the server graph the import evaporates. In the client graph it explodes at build time with a message naming the file.

There's a `client-only` package doing the mirror image, for modules that touch `window` or `document` and should never end up in a server render.

One line at the top of any module holding secrets, database access, or server-side keys. Cheap insurance against the silent version of the failure.

Where you put the directive decides what you ship

Here's the part with real consequences.

Every module reachable from a "use client" entry point ends up in the client graph. Not just the component with the directive — everything it imports, and everything *those* import, all the way down.

So the directive isn't a label on one file. It's the root of a subtree.

Put it at the top of a layout that wraps your whole app, and every component beneath it is client code, in a chunk, downloaded by every visitor. The app still works. Nothing errors. You've just quietly shipped your entire application to the browser and given up most of what RSC was for.

The barrel file version

Chapter 4 covered why barrel files defeat tree shaking. There's a sharper version of the problem at the boundary.

```
// components/index.ts
export * from './Button'      // "use client"
export * from './Modal'      // "use client"
export * from './DataTable'   // "use client"
// ...forty more
```

```
// app/page.tsx — server
import { Button } from '@components'
```

You imported one button. But that index module is now an entry into the client graph, and depending on how much the bundler can prove is unused, you may have dragged a large part of that directory in behind it.

Import from the module directly and the problem disappears:

```
import { Button } from '@components/Button'
```

Push the boundary down

The general fix is to make the client subtree as small as it can be. A page with one interactive button doesn't need to be a client component — the button does.

```
// ✗ everything below is client
'use client'
export default function Page() {
  const [open, setOpen] = useState(false)
  return (
    <article>
      <ExpensiveMarkdownRenderer content={content} />
      <button onClick={() => setOpen(!open)}>Toggle</button>
    </article>
  )
}
```

```
// ✓ only the button is client
export default function Page() {
  return (
    <article>
      <ExpensiveMarkdownRenderer content={content} />
      <ToggleButton />
    </article>
  )
}
```

Same UI. In the second version the markdown renderer and its dependencies never enter the client graph.

Children, and the bundling reason it works

Chapter 3 covered why you can pass a Server Component into a Client Component as children — the server already rendered it, so what arrives is finished output. Here's the bundling half of that answer.

```
// server
<ClientShell>
  <ServerContent />
</ClientShell>
```

`ClientShell` never imports `ServerContent`. There's no import statement, so there's no edge in the graph, so `ServerContent` is never reachable from a client entry point. It stays out of the client bundle entirely.

That's why the pattern isn't a workaround. It's the boundary working exactly as designed: **imports pull things into your bundle; props don't.**

Remember that sentence. It comes back in Chapter 11, where the same distinction decides what enters a cache key and what passes through untouched.

Third-party packages with no directive

A library published before RSC, or one that just hasn't updated, has no "use client" anywhere. If it uses hooks or browser APIs and you import it from a Server Component, the build fails.

Wrap it:

```
// components/Carousel.tsx
'use client'
export { Carousel } from 'some-carousel-library'
```

Now your one-line module is the boundary, and the library sits behind it in the client graph. Import your wrapper from server code and everything resolves.

You'll write a few of these. It's a normal part of an App Router codebase, not a sign anything is wrong.

Go and look

Twenty minutes on your own app, and it will teach you more than the rest of this chapter.

Find your client entry points. Search the repo for "use client". Look at where each one sits in the tree — is it a leaf, or is it wrapping half your app?

Check for a boundary that's too high. Any directive in a layout.tsx, or in a page component that mostly renders static content, is a candidate. Ask what's underneath it.

Look for a client component importing something that touches your server. Database clients, API keys, anything reading process.env at module scope.

Then add import 'server-only' to those modules and rebuild. If the build breaks, you've found a real leak, and you found it before a user did.

You now have the full picture of how code gets divided: what runs where, when it runs, and how the pieces find each other.

What's missing is *time*. Everything so far has been described as if the server produces a payload in one go. It doesn't — it can send

a tree with pieces still missing, and fill them in later on the same response.

That's Suspense, and almost everything people believe about it is wrong.

References

- Dan Abramov — *What Does “use client” Do?*
- Dan Abramov — *Why Does RSC Integrate with a Bundler?*
- Dan Abramov — *How Imports Work in RSC*

Chapter 6 — Suspense Is Not a Loading Spinner

One question before we start.

When a Suspense boundary shows its fallback and then swaps in the real content, how many network requests happened?

“Two” is the common answer, and it’s the answer that makes every decision about boundary placement wrong.

One. The response never closed.

That single fact is the difference between treating Suspense as a loading-state API and understanding it as what it actually is: a transport primitive that lets the server send you a tree with pieces missing and fill them in later.

The API is genuinely misleading here. It’s called Suspense, it takes a prop called `fallback`, and everyone puts a spinner in it. All of that points at “loading UI.” Almost none of it is about loading UI.

What suspends

Three things, and they’re less varied than they look.

An `async Server Component` that hits an `await`. The renderer reaches your component, the component asks for data, the data isn’t there yet. That component has suspended.

A `Client Component` reading an unresolved promise with `use()`. Same shape, different graph.

`lazy()`, while its chunk downloads. Chapter 4’s dynamic import boundary — the code isn’t there yet, so the component can’t render yet.

In every case the pattern is identical: *React needed something to continue and didn't have it*. Suspense is the mechanism for answering the question “so what do we do in the meantime?”

The boundary isn't attached to any of these. It's attached to the tree. React walks up from whatever suspended until it finds the nearest `<Suspense>`, and that boundary's fallback is what gets shown. Which is why placement is a real decision and not decoration.

What actually goes over the wire

Chapter 3 established that the server produces a payload — rows of text describing a tree — and that HTML is generated from it. Both of those can be sent *incomplete*.

When the renderer hits a boundary whose contents have suspended, it doesn't wait. It emits the fallback, writes a placeholder where the real content will go, and carries on rendering the rest of the page. The response stays open.

When the promise resolves, the server writes more rows into that same open response — the real content, and a note about which placeholder it belongs to.

That's it. That's streaming. **The response is a stream that stays open, not a document that gets sent.**

On a first page load there's an extra step, because the browser has HTML on screen and no React yet. React handles this by shipping the late-arriving markup with a tiny inline script that swaps it into place. It runs before hydration, before your bundle has finished downloading. Which is why streamed content appears on slow connections even when the page isn't interactive yet.

Worth watching once. Open the Network tab on a throttled connection and load a page with a slow boundary. You'll see one request, staying open, with bytes arriving in waves.

On a navigation there's no HTML at all — Chapter 3's asymmetry — so the rows just arrive on the open response and React reconciles them in.

No second request. Not on first load, not on navigation.

Every mental model that assumes otherwise leads to putting too few boundaries in, out of a fear of round trips that don't exist.

The same boundary, two transports

Here's the part that isn't in any tutorial.

A `<Suspense>` boundary is one piece of JSX. But what happens when it suspends depends entirely on how the user got to the page — and Chapter 3 already told you why. A document request produces HTML and a payload. A navigation produces a payload only.

So the same boundary streams two completely different ways.

On a first load

There's HTML on screen and no React yet. The browser has painted your fallback and has no idea anything else is coming.

So when the content resolves, the server can't just send a payload row — nothing is listening for one. It sends **markup**, parked in a hidden element at the end of the document, along with a tiny inline script that moves it into the right place and removes the fallback.

That script isn't your bundle. It's a few hundred bytes React inlines into the document, and it runs the moment it arrives.

Which means the swap happens **before hydration**. Before your JavaScript has finished downloading, before React has started, before anything is interactive. On a slow connection you can watch skeletons fill in on a page where clicking still does nothing.

Throttle to Slow 3G once and watch it. Most people have never separated “content arrived” from “page

became interactive” with their own eyes, and the two are genuinely different moments.

On a navigation

No document, no HTML, no inline scripts. React is already running and already holds the tree.

The boundary suspends, the fallback renders from the payload, and when the content resolves the server writes more rows onto the same open response. React reconciles them in — the same reconciliation from Chapter 2, applied to rows arriving over time.

Same JSX. Same boundary. One is a DOM-patching script that runs before React exists; the other is a tree diff inside a React that’s been running for ten minutes.

That’s the thing to hold onto about Suspense. It’s not a component that shows a spinner. It’s a declaration that a subtree is allowed to arrive late — and the machinery for “arriving late” is completely different depending on whether React is awake yet.

Two mechanics worth knowing

loading.tsx is a Suspense boundary at segment granularity. Next wraps the segment in one for you. It’s why most App Router apps have streaming without anyone having decided to use it — and why the granularity is usually coarser than you’d have chosen.

Siblings render concurrently; parent-to-child doesn’t. Two async Server Components beside each other fetch in parallel with no effort from you. But a parent that awaits before rendering a child that also awaits is strictly sequential, because the child cannot start until the parent returns. That’s a property of the renderer, not of your boundaries — and no amount of `<Suspense>` will change it.

Boundaries are also hydration units

A second job, less obvious than the first.

Chapter 3 covered hydration — React executing client components in the browser and attaching handlers to existing DOM. That doesn't happen as one atomic pass. React hydrates boundary by boundary.

Which means it can prioritise. If a user clicks something inside a boundary that hasn't hydrated yet, React hydrates *that* boundary first, ahead of whatever it was going to do next, and then replays the click.

So a boundary does three things at once: it decides what streams separately, what shows a fallback, and what hydrates as a unit. Placement affects all three.

What streaming costs you

Once the first byte is out, headers are gone. That's not React's rule, it's HTTP's, and it has consequences people hit in production.

You cannot change the status code. If your page starts streaming and then a component inside a boundary calls `notFound()`, the response is already a 200. The framework handles the UI — the error surfaces client-side and the right thing renders — but the status code that went out is the status code you're stuck with. Crawlers and monitoring see a 200.

Redirects get messy for the same reason. A redirect after the shell has flushed can't be an HTTP redirect. It becomes a client-side navigation.

The practical shape: anything that needs to affect the *response* rather than the *page* — auth checks, 404s, redirects — should happen before you start streaming. Above the boundary, not inside it.

The sibling primitive: error boundaries

Suspense answers “what if this isn’t ready yet.” There’s an obvious second question — what if this fails? — and it has a nearly identical answer.

```
app/dashboard/  
  error.tsx      ← error boundary  
  loading.tsx    ← suspense boundary  
  page.tsx
```

Same shape, same placement logic, same trade-off. Put the boundary too high and one failed widget takes down the page. Put it around the widget and the rest survives.

The two pair up naturally, because they’re answers to the two ways an async subtree can fail to produce content:

- **Suspense** bounds latency — it isn’t here yet
- **Error boundary** bounds failure — it isn’t coming

And the streaming caveat applies to both. If a component throws *after* the shell has flushed, the status code has already gone out as a 200. The error boundary still renders, client-side, from the stream. But the response was never a 500.

Everything so far has been the read path. A request arrives, the server produces a tree, the browser renders it, and later fills in the parts that weren’t ready.

Nothing has gone the other direction. No form has been submitted, nothing has been written, and the browser has had nothing to say to the server except “give me the next route.”

That changes next. And it changes in the direction you’d expect from Chapter 5: the client gets a reference to a server function, and calling it sends a payload *up* instead of down.

References

- Dan Abramov — *Progressive JSON*
- React docs — `<Suspense>`, `use()`
- Next.js docs — Loading UI and Streaming

Chapter 7 — A Mutation, End to End

Chapter 5 made a claim that should have bothered you.

Functions can't cross the boundary. Not because React is fussy, but because the two sides never share a scope — there's no lexical environment on the client for a server closure to be captured from. Props go into a text payload. A function isn't text.

And yet:

```
// server
async function likePost(id: string) {
  'use server'
  await db.likes.create({ postId: id })
}

// client
<LikeButton onLike={likePost} />
```

That works. A function, passed across the boundary, called from the browser.

The resolution is the same trick as Chapter 5, pointed the other way. **The function doesn't cross. A reference does.**

What "use server" compiles to

Chapter 5: when the bundler builds the *server* graph and finds "use client", it replaces the module with a stand-in carrying an address.

This is the mirror. When the bundler builds the **client** graph and finds "use server", it replaces the function body with a stub. The stub

holds an ID — a deterministic hash of the file path and the export name, generated at build time the way Chapter 4’s module IDs were.

Your server code is not in that stub. It was never in the client graph at all.

Calling it does this:

```
POST /whatever-route-you-are-on
Next-Action: 7f2a9c4e1b...
[serialized arguments]
```

An HTTP request to your own origin, with an action ID in a header and the arguments in the body. The server looks the ID up in a manifest — a second one, the mirror of Chapter 5’s client manifest — finds the real function, and calls it.

So a Server Action is an RPC endpoint that you didn’t write a route for. The framework generated the ID, the route, the serialization and the dispatch. You wrote a function and a directive.

A detour: what RPC actually means

Worth unpacking, because the term gets used casually and it’s the whole shape of what’s happening here.

RPC is Remote Procedure Call. The idea is old — 1980s old — and the goal has always been the same: make calling code on another machine look exactly like calling a function locally. You write `deletePost(42)`. Something serializes the argument, sends it over a network, runs the function elsewhere, serializes the result, sends it back. The remoteness is hidden on purpose.

The useful contrast is REST. **REST models nouns:** you have resources, and you act on them with a small fixed set of verbs. `DELETE /posts/42`. **RPC models verbs:** you have operations, and you call them. `deletePost(42)`. Neither is better; they’re different framings of the same network call. REST won on the public web because resources and HTTP verbs map neatly onto caching, proxies and

URLs. RPC never went away for service-to-service work, where you don't need any of that and you do want a function call.

gRPC is the modern version you've probably heard of. Google's, from 2015, and it works like this: you write a schema file describing your service and its methods, a code generator turns that schema into client and server stubs in your language, and you call methods on the stub. The stub handles serialization — Protocol Buffers, a compact binary format — and transport over HTTP/2. From your code's point of view, it's a method call.

Now line that up against what you just read.

- gRPC: you write a schema, a **code generator** produces a stub, calling the stub does the network work
- Server Actions: you write a directive, the **bundler** produces a stub, calling the stub does the network work

Structurally the same thing. And the difference is the part that matters.

gRPC has a contract. Server Actions don't. That schema file is enforced at both ends — the generated server stub rejects a payload that doesn't match the declared types before your code ever sees it. A Server Action has no schema. The bundler generated an ID and a dispatcher, and nothing anywhere describes what a valid call looks like. Your function signature says (`id: string`), but TypeScript is erased at build time. What actually arrives is whatever the caller sent.

That's why the next section exists, and it's why validation inside the action isn't defensive programming. It's the only contract there is.

There's an older lesson under this too. RPC's history is a long series of attempts to make the network invisible, and the recurring result is that you can't. Remote calls fail in ways local calls don't, take time local calls don't, and — the one people forget — can be made by anyone, not just by your code. Every RPC system eventually rediscovers this. Server Actions are the newest and most ergonomic instance, and the ergonomics are exactly what make it easy to forget.

Every action is a public endpoint

That last sentence has a consequence, and it is the single most important thing in this chapter.

The action ID is in your client bundle. It's in the HTML. Anyone can read it, and anyone can POST to it, with any arguments they like, from anywhere.

Which means this is not protection:

```
export default async function Page() {
  const user = await getUser()
  return (
    <>
    {user.isAdmin && <DeleteButton onDelete={deletePost} />}
    </>
  )
}
```

Hiding the button hides the button. The action still exists, still has an ID, and still runs for anyone who sends the request. You have written an unauthenticated delete endpoint and put a conditional render in front of it.

Every action needs its own check, inside the function:

```
async function deletePost(id: string) {
  'use server'
  const user = await getUser()
  if (!user?.isAdmin) throw new Error('Unauthorized')
  await db.posts.delete({ id })
}
```

The mental shift: **stop thinking of an action as a function you're calling and start thinking of it as a route you're**

publishing. You wouldn't ship an API route with no auth check because the UI doesn't link to it. This is the same thing with better ergonomics and worse instincts around it.

This is also the other half of the point Chapter 3's appendix made about `proxy.ts`. A proxy that gates *pages* does nothing here, because an action is never routed through a page request — the POST goes to the action ID directly. Centralising auth at the proxy guards the front door and leaves every action door unlocked.

This surface has already produced real vulnerabilities — CVE-2025-55182, a remote code execution bug in the Flight deserialization path, and a separate denial-of-service issue. The attack surface is exactly what you'd expect once you accept that your app accepts serialized data at generated endpoints.

What travels with the call

The arguments are serialized, with the same rules as props from Chapter 3. Primitives, plain objects, arrays, Date, Map, Set, FormData. Not functions, not class instances.

Then there's the part that catches people.

```
export default async function Page() {
  const apiKey = process.env.SECRET_KEY

  async function doThing() {
    'use server'
    await callApi(apiKey) // closes over a server value
  }

  return <ClientForm action={doThing} />
}
```

An inline action defined inside a component closes over that component's scope. But the action is called from the browser, which

doesn't have that scope — so the closed-over values have to travel *with the reference*, out to the client and back.

Next encrypts them, so it isn't a plaintext leak. But they're still crossing the boundary, on every render, and the encryption is the only thing between that value and a bundle someone can download. Treating an inline closure as "server-side, therefore safe" is the wrong instinct.

Read the environment variable inside the action instead of closing over it. Costs nothing, and the value never leaves.

Forms work before your JavaScript does

```
<form action={createComment}>
  <textarea name="body" />
  <button>Post</button>
</form>
```

Chapter 3 covered the hydration gap — HTML painted, nothing interactive yet. Chapter 6 covered content arriving before React wakes up. This is the third instance of the same theme.

If a user submits that form before hydration finishes, it still works. The browser does a native form POST to the current URL, the framework routes it to the action by ID, and the response comes back as a full document.

After hydration, React intercepts the submit and does it as a fetch instead. Same action, same code, different transport.

That's real progressive enhancement, and you get it by using action instead of onSubmit. Reach for onSubmit and a handler and you've opted out of it without noticing.

The response is a tree

Here's the payoff, and it's the thing that makes Server Actions different from every RPC layer you've used before.

The POST returns a **Flight payload**. Not JSON.

It contains the action's return value, and — if the action revalidated anything — a freshly rendered tree for the affected route. React reconciles that tree into the page.

So the full loop is:

1. Client calls the reference
2. POST goes out with the ID and arguments
3. Server runs the function, writes to the database
4. `revalidatePath` or `revalidateTag` marks the affected route stale
5. **The server re-renders it, in the same request**
6. The response carries the return value *and* the new tree
7. React reconciles

One round trip. Not “mutate, then refetch.” The mutation response *is* the refetch.

That's why Chapter 3 said mutations return trees. Everything since has been building toward the fact that a tree is a thing you can send over a wire, and this is where that pays off — the server can reply to a write with a new view of the world, and the client can just diff it in.

`revalidatePath` and `revalidateTag` are the trigger for step 5. What they actually invalidate, and what “stale” means across the four caching layers, is Chapter 11. Here they're just the switch that says *this route needs re-rendering before you reply*.

The hooks

Four of them, each solving a specific problem that falls out of the above.

useActionState wraps an action and gives you back the last result and a pending flag. It's the one to reach for by default, because it keeps working before hydration — the form still submits natively, and the state hooks up once React is running.

useFormStatus reads the pending state of the nearest enclosing form. It has to be called from a component *inside* the `<form>`, not the one that renders it, which is a constraint people trip on constantly. If it always reports false, that's why.

useTransition for actions not attached to a form — a like button, a delete. Gives you a pending flag without a form to read from.

useOptimistic shows the expected result immediately and reconciles when the real one lands. The part worth knowing: you don't write the rollback. The optimistic value is discarded automatically when the action settles and the real state arrives. If the action failed, the real state is the old state, and the UI reverts on its own.

Redirects and errors

`redirect()` works by throwing. That's not a quirk to work around, it's the mechanism — it throws a special error the framework catches and turns into a redirect.

Which means this breaks:

```
async function createPost(data: FormData) {
  'use server'
  try {
    await db.posts.create(...)
    redirect('/posts')          // throws
  } catch (e) {
    return { error: 'Failed' } // swallows the redirect
  }
}
```

The redirect never happens, and the user gets a generic error on a post that saved fine.

The fix isn't a Next.js API. It's the general rule that a broad catch is almost always a bug: **catch what you expect, not everything**. This is the same failure as swallowing an `AbortError` or a cancellation signal — control flow passing through a try that had no business intercepting it.

So narrow the try to the thing that can actually fail:

```
async function createPost(data: FormData) {  
  'use server'  
  
  let post  
  try {  
    post = await db.posts.create({ ... })  
  } catch (e) {  
    return { error: 'Could not save your post' }  
  }  
  
  redirect(`/posts/${post.id}`) // outside the try  
}
```

That's the answer in nearly every case, and it makes the code better on its own terms — the try now wraps one fallible operation instead of a whole function body.

If you genuinely can't restructure — a shared error-reporting wrapper, a helper you don't control — Next provides an escape hatch that lets framework errors pass through:

```
import { unstable_rethrow } from 'next/navigation'  
  
try {  
  await doTheThing()  
  redirect('/posts')  
} catch (e) {
```

```
// framework errors (redirect, notFound) pass through
unstable_rethrow(e)
return { error: 'Could not save your post' }
}
```

Note the prefix. It's `unstable_` because the underlying mechanism — control flow via thrown sentinels — is something React and Next want room to change. Treat it as a patch for a try block that's too wide, not as the standard way to write actions.

`notFound()` works the same way and breaks in the same place, so a catch that swallows one swallows both. If you have a broad catch anywhere near navigation calls, that's where to look first.

For real errors: in production, an error thrown from an action reaches the client as a generic message with a digest ID, not your actual error text. That's deliberate — action errors can contain database messages, file paths, query fragments. If you want the user to see something specific, **return it rather than throwing it**. Thrown means unexpected; returned means handled.

That's the loop closed. A request comes in, the server produces a tree, the browser renders it, streams what wasn't ready, and can now send data back and receive a new tree in reply.

Which raises the question this whole book has been walking toward. If the server owns the data, the fetching, the mutations and the re-rendering — what is actually left for the client to hold?

Less than you think, and not the things you'd guess.

References

- Dan Abramov — *What Does “use client” Do?*
- React docs — `Server Functions`, `useActionState`, `useOptimistic`
- Next.js docs — `Server Actions and Mutations`

Chapter 8 — What’s Actually Left for the Client

This book has spent five chapters taking things away from the browser.

Rendering moved to the server. Data fetching moved to the server. Mutations moved to the server, and Chapter 7 established that they come back as a fresh tree rather than as JSON you have to merge yourself.

So what’s left?

Less than you’d guess, and — the more useful part — not the things you’d guess.

useState is a default, not a decision

Most of us learned React in an order that made `useState` mean “state.” It was the first hook you saw, it was in every tutorial, and everything that changed over time went in one.

That was reasonable when the browser owned everything. It isn’t now.

State has **four possible owners**, and `useState` is one of them. Picking the wrong owner doesn’t usually produce an error — it produces an app that’s subtly wrong in ways nobody files a bug about. A filter that vanishes on refresh. A link that doesn’t reproduce what the sender was looking at. Two values that disagree.

Owner 1: the server

Your data. Posts, users, prices, orders. It lives in a database and the server can read it.

If you're holding server data in `useState`, you have built a cache. Not metaphorically — an actual copy of remote data, in memory, on one machine, and now you own every hard problem that comes with one. When does it go stale? What happens if two tabs disagree? What refreshes it after a mutation?

Chapter 7 answered that last one: nothing needs to, because the mutation response already contains the new tree. The moment you copy that data into `useState`, you've opted out of the mechanism and taken the job back.

```
// ✗ a cache with no invalidation strategy
'use client'
function Posts() {
  const [posts, setPosts] = useState([])
  useEffect(() => {
    fetch('/api/posts').then(r => r.json()).then(setPosts)
  }, [])
}
```

```
// ✓ the server owns it
async function Posts() {
  const posts = await db.posts.findMany()
  return <PostList posts={posts} />
}
```

The second version has no loading state, no error state, no stale data and no effect. Those weren't features you gave up. They were costs you were paying for a copy you didn't need.

Owner 2: the URL

Anything that should survive a refresh, be shareable, or work with the back button.

Filters. Sort order. Pagination. Search queries. Which tab is active. Sometimes which item is selected.

The test is simple: **if a user sent this page to a colleague, should the colleague see the same thing?** If yes, it belongs in the URL. Put it in `useState` and you've built an app where the address bar lies.

```
// ✗ the URL says /products regardless of what you're looking at
const [category, setCategory] = useState('all')
```

```
// ✓ the URL is the state
const params = await searchParams
const category = params.category ?? 'all'
```

Back and forward work for free, because they were always going to — you just weren't using them.

Two honest costs. In an App Router app, changing a search param is a navigation, so it round-trips to the server unless you deliberately avoid that. And `searchParams` is a Promise in Next 16, which makes the segment dynamic. Neither is a reason to keep filters in local state; both are reasons to know what you're choosing.

Owner 3: the form

The DOM already stores what's in your inputs. It's been doing that since 1995.

```
// ✗ React re-implementing what the input already does
const [email, setEmail] = useState('')
const [password, setPassword] = useState('')
<input value={email} onChange={e => setEmail(e.target.value)} />

// ✓ read it when you need it
<form action={signIn}>
  <input name="email" />
  <input name="password" type="password" />
</form>
```

Chapter 7 covered where the values go — FormData, into the action. And `useActionState` holds the result, so the outcome of the submit isn't your state either.

Controlled inputs aren't wrong; they're just not the default. Reach for them when you need something *during* typing: live validation, a field that depends on another field, formatting as the user types, a character counter. If nothing needs to happen until submit, nothing needs to be in React.

Owner 4: ephemeral UI state

This is the real `useState` territory, and it's smaller than the space `useState` currently occupies.

Is the dropdown open. Is the tooltip showing. Which row is hovered. Is the accordion expanded.

Three properties, all of which have to hold:

- Meaningless outside this component
- Nobody should be able to link to it
- It *should* disappear on refresh

That's it. That's what local state is for.

Where does this state actually live?

Worth being concrete, because it explains several things you already rely on without thinking.

React keeps a **fiber** for every component instance — a plain JavaScript object holding what React knows about that component. Attached to it is a linked list of hook records, and `useState` stores its value in one of those.

So state is an object on the JavaScript heap, in that tab's process. RAM, and nothing more durable.

Which is where its properties come from:

- **Refresh and it's gone.** New heap, new fibers, initial values.
- **Two tabs have two separate states.** Separate processes, separate heaps. They can't see each other and never could.
- **Nothing is persisted anywhere** unless you wrote code to persist it.
- **It doesn't exist on the server.** Chapter 3 covered the SSR pass — `useState` returns its initial value and nothing updates it, because there's no interaction and no heap that survives the request.

Context lives the same way — on the provider's fiber, and consumers read it by walking up the tree to find it. How a change to that value propagates is a Context question specifically, and it's covered where Context is.

The audit

Take a real client component and ask, for each piece of state, who should own it.

```
'use client'
function ProductBrowser() {
```

```
const [products, setProducts] = useState([])
const [category, setCategory] = useState('all')
const [query, setQuery] = useState('')
const [sortBy, setSortBy] = useState('newest')
const [filtersOpen, setFiltersOpen] = useState(false)

useEffect(() => {
  const q = `category=${category}&q=${query}&sort=${sortBy}`
  fetch(`/api/products?${q}`)
    .then(r => r.json())
    .then(setProducts)
}, [category, query, sortBy])
}
```

Five pieces of state. **One of them belongs here.**

- products — server-owned
- category, query, sortBy — URL-owned, all three
- filtersOpen — ephemeral, correctly placed

Re-file the other four and the component stops being a client component at all, except for the small piece that opens a panel. The effect disappears. The loading state disappears. The URL becomes honest.

Two things that aren't state

Derived values. If you can compute it from state you already have, computing it is not an optimization — it's a correctness requirement.

```
// x two sources of truth that can disagree
const [items, setItems] = useState([])
const [total, setTotal] = useState(0)
```

```
// ✓ one source of truth
const [items, setItems] = useState([])
const total = items.reduce((sum, i) => sum + i.price, 0)
```

The first version has a bug waiting in it: any code path that updates items without updating total. The second version cannot have that bug. Chapter 2 covered why the recomputation almost certainly doesn't matter.

Values that change without affecting the screen. A timer ID, a previous value you're comparing against, a DOM node, a flag saying something already ran.

If reading it doesn't change what's rendered, it's a ref. `useState` for these means re-rendering to store a value nobody displays.

useReducer, for state that moves together

When several values must change as a unit, separate `useState` calls let them drift into combinations that should be impossible — loading and error both true, data present while still loading.

A reducer makes the transitions atomic and gives them names. `dispatch({ type: 'submit_failed' })` says what happened; three `setState` calls in a row say what you did about it.

This isn't Redux-in-miniature. There's no store and nothing is global. It's the same local state with the transitions written down.

Context is not a state manager

Context is dependency injection for the tree. It answers "how does something deep down get a value from up high without threading it through ten components."

It's good at values that rarely change: theme, locale, a configured client, the current user. It's bad at values that change often, because every consumer re-renders when the value changes — and that's not a tuning problem, it's what Context does.

One RSC-specific point worth having straight, because people get it backwards.

A Context provider has to be a Client Component. That leads to a common fear: putting a provider near the root makes your whole app client-side.

It doesn't — for the reason Chapter 5 gave. The provider's children arrive as children, which is a prop, not an import. No import edge, no path into the client graph.

```
// app/layout.tsx – still a Server Component
export default function Layout({ children }) {
  return <ThemeProvider>{children}</ThemeProvider>
}
```

ThemeProvider is client. Everything inside children is whatever it was already. The boundary doesn't spread downward through props — only through imports.

Does a context change re-render the whole tree?

No — but the useful answer is more precise than that, because two different mechanisms get blamed for the same symptom.

One: the provider re-rendered. If the provider component's own state changed, it re-renders, and its children re-render as a normal parent-child consequence. This has nothing to do with context specifically. It's avoidable the usual ways — passing children through as a prop, or memoizing.

Two: the context value changed. Now every component calling `useContext` for that context re-renders. Not the whole subtree — only consumers. But here's the part that matters: **memo does not stop this**. Context propagation deliberately bypasses

memoization, because a memoized component that ignored a context change would render stale data.

That second mechanism is the real argument against putting frequently-changing values in Context. It isn't that it re-renders too much — it's that consumers have no way to opt out. A value that changes on every keystroke re-renders every consumer on every keystroke, and no amount of memoization at the call site will help. If that's the situation you're in, the value doesn't belong in Context; it belongs in a store built for granular subscription — which is exactly the question the next chapter takes up.

useSyncExternalStore, the honest escape hatch

Sometimes a value lives outside React and React needs to read it: online/offline status, a media query, the contents of `localStorage`, a WebSocket, a store from a library.

These have something in common — nobody told React they changed. React finds out only if something explicitly tells it.

The obvious approach is an effect that subscribes and calls `setState`:

```
// x
const [isOnline, setIsOnline] = useState(true)
useEffect(() => {
  const on = () => setIsOnline(true)
  const off = () => setIsOnline(false)
  window.addEventListener('online', on)
  window.addEventListener('offline', off)
  return () => { /* remove both */ }
}, [])
```

This mostly works, and it fails in two specific ways.

Tearing. React can render in pieces, pausing between components. If the external value changes *during* a render

pass, components rendered before the change read the old value and components rendered after read the new one. One frame, two answers, from the same source. That’s tearing, and it’s invisible in testing and maddening in production.

Server rendering. `window` doesn’t exist on the server. So the initial value is a guess — here, `true` — and if the guess is wrong, hydration mismatches.

The built-in solves both:

```
const isOnline = useSyncExternalStore(
  (callback) => {
    window.addEventListener('online', callback)
    window.addEventListener('offline', callback)
    return () => {
      window.removeEventListener('online', callback)
      window.removeEventListener('offline', callback)
    }
  },
  // read the current value in the browser
  () => navigator.onLine,
  // read it during server render
  () => true
)
```

Three arguments, and each maps to one of the problems:

- **subscribe** — how React gets told the value changed. It hands you a callback; you call it whenever the source changes.
- **getSnapshot** — how React reads the current value *right now*, synchronously. Because it’s a synchronous read rather than a copy in state, React can force every component in a pass to see the same value. That’s the tearing fix.
- **getServerSnapshot** — what the value is during server rendering, where `navigator` doesn’t exist. This is the argument that confuses people: it isn’t doing anything *with* your server.

It's answering "what should this be when there's no browser," so the SSR pass and hydration agree.

Worth knowing for two reasons. It's the correct primitive for browser APIs, which is most of what people reach for effects to do. And it's what modern state libraries subscribe through — so when you look at one, you're largely looking at ergonomics layered over this.

Chapter 10 comes back to this. Most of what people write effects for is this problem, and this hook is the answer to it.

So what's actually left?

Run the audit across a normal CRUD application and the answer is: surprisingly little, and almost none of it global.

Data is the server's. Anything shareable is the URL's. Form values are the form's. What remains is a scatter of small, local, ephemeral flags — open, hovered, expanded — that were never global to begin with.

That's not an argument that global client state is unnecessary. It's an argument that most of what people put in a global store was never client state at all, and that once you stop miscategorising, the pile is much smaller than the tooling around it suggests.

But it isn't zero. Some applications genuinely have a lot of client state, with complicated transitions between pieces of it — editors, canvases, dashboards, anything with undo/redo, anything that works offline. For those, "just use `useState`" stops being an answer and a store becomes a real question.

That question deserves its own chapter: why a store has one of them rather than many, what problem the single-store design was actually solving, and how to tell whether you have that problem.

References

- [React docs — Choosing the State Structure, useSyncExternalStore](#)
- [Next.js docs — searchParams](#)

Chapter 9 — What Redux Was Actually For

The last chapter argued that most of what people put in a global store was never client state — it was server data, or URL state, or form values, all miscategorised.

That argument has a limit. Some applications genuinely have a lot of client state — editors, canvases, dashboards with cross-filtering, anything with undo, anything offline-first — and for those, “just use `useState`” stops being an answer.

Which is usually the point someone reaches for Redux, carrying a set of beliefs about it that mostly don’t survive contact with why it was built. Worth taking apart one at a time.

Misconception 1: “It’s a state manager”

Not really — it’s an **event log with derived state**. The store isn’t the primary thing; the sequence of dispatched actions is. State is just what you get from replaying that sequence from the start.

That framing sounds abstract until you see what it was replacing.

What 2015 looked like

Redux’s design makes very little sense unless you remember what it was competing with.

Prop drilling was the only option. Context existed but was undocumented, marked experimental, and warned against. Getting a value from the top of the tree to the bottom meant threading it through every component in between, whether they cared or not.

There was no server-state story. No RSC, no query libraries. If your app needed data, you fetched it into component state and then dealt with every consequence yourself.

Nobody could explain why the UI was wrong. Two-way binding and scattered mutation meant state changed from many places, at many times, for reasons that were hard to reconstruct after the fact.

That third one is the important one. Redux was not primarily a solution to prop drilling — Context solved that. Redux was a solution to **not being able to explain how the app got into this state.**

Misconception 2: “The single store is an arbitrary rule”

It isn’t arbitrary — it’s the price of the thing misconception 1 just described.

Once you decide the goal is explicability, the design mostly writes itself.

If state is in one object, the app has one state. Not a set of overlapping fragments that might disagree — one value you can log, serialise, and inspect. “What is the app’s state right now” has an answer you can print.

If every change is a described action, history is a list. { type: 'todo/added', payload } is a record of what happened, not of what you did about it. Replay the list from the initial state and you’re back where you were.

Which makes time-travel debugging possible. State plus an ordered action list means you can step backwards and forwards through the app’s history. That was the demo that sold Redux, and it does not work with state scattered across many independent stores — there’s no single timeline to travel along.

And updates are atomic. One dispatch, one new state object, one notification. Two pieces of related state can’t be briefly inconsistent because they were updated by two separate calls.

The single store isn't dogma or taste. It's what "the app has an inspectable, replayable history" costs.

Misconception 3: "Global state means Redux"

Redux calls itself a library for global state management, and it is one. The mistake isn't thinking it's for global state — it's thinking *global* is the property that tells you to reach for it.

It isn't. Plenty of global state has no business in Redux. A theme, a locale, the current user — global, and perfectly well served by Context. And a great deal of what people globalised *into* Redux was server data that was never client state in the first place, just cached remote data wearing a store's clothing.

The property that actually distinguishes Redux-shaped problems is **traceability** — state whose changes you need to log, inspect and replay, exactly the event-log design from the first misconception. Some global state needs that. Most doesn't. "Is it global" and "does it belong in Redux" are different questions, and conflating them is how a store ends up holding your entire app.

Which is visible once you look at what took each job away.

Almost every job Redux was doing in a typical CRUD app has been taken by something more specific.

Server data → the server. The largest use of Redux by volume was caching API responses: a posts slice, a users slice, loading flags, error flags, normalised entities, and reducers to keep it coherent. RSC removed the need for the copy, and Chapter 7 established that mutations return a fresh tree rather than data you merge yourself. For apps that still fetch on the client, query libraries do this job better because they were designed for exactly it — caching, deduplication, revalidation, staleness.

Shareable state → the URL. Filters, tabs, pagination, search. Chapter 8's argument: if a colleague should see the same thing when you send them the link, it's URL state, and it never needed a store.

Passing values down → Context. Documented, stable, and adequate for values that rarely change. The original prop-drilling problem is just solved now.

Coordinated local state → useReducer. Actions and reducers, minus the store and the global scope. If you liked Redux’s shape but never needed the globalness, this was always the thing you actually wanted.

What’s left after those four is the honest scope of the question — and it’s smaller than “global state” ever was.

Misconception 4: “Reducers are just ceremony”

They read that way until you need them. A reducer’s actual job is making impossible states unrepresentable — `{ status: 'loading', data: null, error: null }` as one value out of a named set, instead of three independent booleans that can disagree.

Once you accept that job, the “one store or many” question turns into a real trade rather than a style argument.

Atom-based state — Jotai, Recoil, and in a different way Zustand — breaks state into many small independent pieces. A component subscribes to the specific atoms it reads, and only re-renders when those change.

The upside is precision. No selectors to write, no worrying that a component re-renders because an unrelated slice changed, no single object growing without bound.

The cost is exactly what the single store bought. There’s no one place to inspect. There’s no single ordered history, so there’s no time travel. Two atoms updated in sequence can be observed in an intermediate state. You’ve traded explicability for granularity — the same trade misconception 1 makes visible.

Chapter 2’s Signals section is the same trade at a different layer — fine-grained subscription instead of coarse re-render, precision in exchange for a single coherent pass you can reason about. Worth

noticing that the industry keeps making that trade in the same direction.

Misconception 5: “Big app, therefore Redux”

Size was never the criterion. Traceability was. A large app made entirely of server data, URL state and forms needs none of this; a small app with a complex undo stack might need all of it. Concrete cases, not vibes:

Complex client state with real transitions. An editor where selection, tool, clipboard, zoom and dirty-status interact. The states that shouldn’t exist are numerous, and a reducer is how you make them unrepresentable.

Undo and redo. If you need history, you need a single ordered list of changes. This is the case Redux’s design was built for and nothing has replaced it.

Offline-first. A local source of truth that syncs when it can. That’s a client-owned data store by definition.

Cross-cutting client state. State that many distant parts of the tree read *and* write, that isn’t server data and can’t live in a URL.

Genuinely hard debugging. If “how did it get like this” is a question your team asks weekly, an inspectable action log is worth its ceremony.

Redux Toolkit is what you’d actually use — the boilerplate that gave Redux its reputation is largely gone.

Misconception 6: “It’s already in the project, so everything goes there”

This is the most common one in practice, and it’s different from the other five — they’re about what Redux *is*, this one is about what

people do once it's installed. The reasoning is seductive: one place for state, one mental model, no per-feature debate about where things live. Consistency is a real virtue. This just spends it on the wrong thing.

Three versions of it, all worth catching.

“Anything used in several places should just go in Redux.”

Being read in many places is a real reason to lift state out of a component — that part's right. It just doesn't say *Redux* on its own, because “used widely” splits three ways. A value read in many places that rarely changes — a user profile, theme, locale — is Context's home turf; a Redux slice for it is a reducer, an action and a selector to move something that changes at login and logout. A value read *and written* in many places, changing often with real transitions, is the actual Redux case. And a value read everywhere that happens to be server data is still server data — the number of consumers doesn't make it client state. The profile is the trap here: it's almost always either cached server data or a stable widely-read value, and both have homes that aren't a store.

“All the localStorage stuff should hydrate into Redux.”

This turns one source of truth into two. The value now lives in `localStorage` *and* in the store, with a write path, a boot-time read path, and a bug waiting for the first time they disagree. `localStorage` already survives refresh — usually the only reason it was picked. Read it where you need it (`useSyncExternalStore` is the honest way) and there's one source of truth. Mirror it into Redux and there are two, plus a sync problem you now own.

“I don't use Context anymore, I have Redux.” These aren't competing tools. Context is dependency injection — it moves a value down the tree. Redux is a state container. They're so much not-in-competition that Redux *uses* Context to hand you the store: `<Provider>` is Context. Dropping Context because you have Redux is dropping a delivery van because you built a warehouse. For a value that changes twice a year — theme, locale — Context alone is less machinery than a slice with its selector, action and reducer.

The error underneath all three is treating “already installed” as

“is the answer.” A tool’s cost isn’t the install. It’s every future reader having to load your entire store into their head to understand one feature. A store holding server data, URL state, form values and localStorage mirrors isn’t a state strategy — it’s a junk drawer with a subscribe method.

The question to ask first

Before choosing a store, categorise what you’re storing. Chapter 8’s four owners: server, URL, form, ephemeral.

If most of it is server data, **you don’t have a state management problem. You have a caching problem** — and a store is the wrong tool, because a store has no opinion about staleness, revalidation, or what happens when two tabs disagree. You’d be building a cache by hand, again, with a state library.

If most of it is genuinely client-owned, with complex transitions between pieces, then the question is real, and it’s the narrower one this chapter has been about: do you need one inspectable history, or many precise subscriptions?

That’s a decision you can actually make. “Should I use Redux” isn’t.

References

- Dan Abramov — *You Might Not Need Redux*
- Redux docs — Motivation, Three Principles, Redux Toolkit

Chapter 10 — Effects Are Not Lifecycle

Chapter 3 established a fact in passing that’s worth picking back up: **useEffect does not run during the SSR pass**. Your client component executes on the server, produces markup, and its effects never fire.

That’s not a limitation. It’s a definition, and it’s the clearest statement of what an effect actually is.

An effect is not “code that runs after render.” It’s a request to **synchronise something outside React with something inside it**. The server has no outside — no DOM, no window, no subscriptions, nothing to keep in sync. So there’s nothing for an effect to do, and it doesn’t run.

“componentDidMount but for hooks” is what most of us were told `useEffect` was, and it’s the source of most of the trouble below.

Where the lifecycle idea came from

Class components had named lifecycle methods. `componentDidMount`, `componentDidUpdate`, `componentWillUnmount`. They were about *the component’s* life: born, changed, died.

When hooks arrived, `useEffect` got explained as the replacement, usually with a translation table. Empty deps means mount. Some deps means update. The return function means unmount.

That table is why so much effect code is wrong. It teaches you to ask *when does this run* when the actual question is *what is this keeping in sync*.

The lifecycle framing survives because it’s approximately true for simple cases and it fails silently for everything else.

The cleanup function is not “unmount”

The most consequential thing in the whole API, and the translation table gets it exactly wrong.

Cleanup does not run “when the component unmounts.” It runs **before every re-run of the effect**, and also on unmount.

So an effect isn’t a thing that happens once. It’s a **setup/cleanup pair that can happen many times**, and each pair has to be self-consistent:

```
useEffect(() => {
  const conn = createConnection(roomId)
  conn.connect()
  return () => conn.disconnect()
}, [roomId])
```

Change `roomId` and React disconnects from the old room, then connects to the new one. It doesn’t matter whether that’s a mount, an update, or the fifteenth change — the pair holds.

Write the same thing with the lifecycle framing and you get a connection that’s established on mount, never torn down when the room changes, and leaked.

The test for a correct effect: could it run twice in a row and leave the world in the same state? If not, the cleanup is wrong or missing.

The clearest case: <Activity>

Chapter 1 mentioned that Next 16 doesn’t unmount the route you navigate away from. It wraps it in React’s <Activity> component in hidden mode, so component state on that page is still there when you go back.

What happens to that page's effects is the interesting part, and it's the sharpest illustration of everything above.

They're cleaned up. Every `useEffect` and `useLayoutEffect` cleanup function runs, exactly as if the component had unmounted. The setup functions run again when the Activity becomes visible. React's own documentation says to think of a hidden Activity as unmounted — the point being that subscriptions shouldn't stay live for UI nobody is looking at.

So state persists and effects don't.

That is only surprising if you still believe cleanup means unmount. If you've taken the setup/cleanup pair seriously, `<Activity>` needs no special handling at all — your effect already tears down and rebuilds correctly, because it was always going to have to. An effect written against the lifecycle model, by contrast, now has a second way to break that didn't exist before.

Which is what StrictMode is for

In development, React deliberately runs setup, then cleanup, then setup again on mount.

People treat this as a nuisance and reach for a ref to suppress it. It's a test. It runs your setup/cleanup pair twice and shows you whether the pair is symmetric. If double-invoking breaks your effect, the effect was already broken — it just needed a second `roomId` change in production to show you.

Suppressing it hides the bug until a user finds it.

Effects you don't need

Most `useEffect` in real codebases isn't synchronising with anything external. Three patterns cover most of it.

Deriving a value. Chapter 8 covered this: if you can compute it, compute it.

```
// x an extra render, and a window where they disagree
const [total, setTotal] = useState(0)
useEffect(() => { setTotal(items.reduce(...)) }, [items])

// ✓
const total = items.reduce(...)
```

Responding to an event. If something should happen *because the user did a thing*, it belongs in the handler, not in an effect watching for the consequence.

```
// x runs whenever cart changes, for any reason
useEffect(() => {
  if (cart.length > 0) showToast('Added to cart')
}, [cart])

// ✓
function handleAdd(item) {
  setCart([...cart, item])
  showToast('Added to cart')
}
```

The first version fires on page load if the cart was restored from storage. It fires when an item is removed. It has no idea *why* the cart changed, because effects can't — they see state, not causes.

Resetting state when a prop changes. There's a built-in mechanism and it isn't an effect:

```
// x renders once with stale state, then again with reset state
useEffect(() => { setDraft('') }, [postId])

// ✓ a new key means a new component with fresh state
<Editor key={postId} post={post} />
```

Chapter 2 covered why keys control identity. This is that mechanism used deliberately.

The deps array is a correctness contract

The lint rule that tells you to add a dependency is not a style preference and it is not about performance. It's telling you your effect reads a value it hasn't promised to stay in sync with.

Remove a dep to “stop it running so much” and you’ve built a stale closure — the effect keeps a snapshot of a value from a render that’s long gone, and it will read that snapshot forever.

The right move is almost never removing a dep. It’s changing what the effect closes over: move the value inside, use the updater form of `setState`, or split one effect into two that synchronise different things.

If the deps array is fighting you, the usual cause is that one effect is doing two unrelated jobs.

`useEffectEvent`, for reading without reacting

There’s a genuine tension the deps array can’t express: sometimes you want to *read* a value without *reacting* to it changing.

```
useEffect(() => {
  const conn = createConnection(roomId)
  conn.on('message', () => showNotification(theme))
  return () => conn.disconnect()
}, [roomId, theme]) // reconnects on theme change. absurd.
```

The effect reads `theme`, so the lint rule wants it in the deps, so changing the theme tears down your connection.

`useEffectEvent` separates the two. You extract the part that reads latest values into a function the effect can call without depending on it:

```
const onMessage = useEffectEvent(() => showNotification(theme))

useEffect(() => {
  const conn = createConnection(roomId)
  conn.on('message', onMessage)
  return () => conn.disconnect()
}, [roomId])
```

Reads the current theme every time, doesn't reconnect when it changes.

useLayoutEffect, and the actual sequence

To place `useLayoutEffect` you need the order of operations, which is worth having anyway.

1. **React render phase.** Your components are called. React builds the new tree and diffs it. Nothing has touched the DOM.
2. **React commit phase.** React applies the changes to the DOM. The DOM now says the new thing — but the screen doesn't yet.
3. **useLayoutEffect runs, synchronously.** The browser is blocked here. You can read layout and write more DOM, and the user will never see the intermediate state.
4. **The browser does its work.** Recalculate styles, run layout, paint, composite. Now the screen changes.
5. **useEffect runs.** After paint, asynchronously.

So the difference is one step: layout effects run before step 4, passive effects after.

That makes `useLayoutEffect` blocking by definition — you’re holding the frame. A bad default, and the reason it isn’t one. Use it only when the user would otherwise see a wrong frame: measuring an element and immediately repositioning something based on the measurement. A tooltip that renders at the wrong spot and then jumps is the canonical case.

It also doesn’t run on the server, which produces a warning if you use it in server-rendered code. That warning is true and worth heeding — measuring requires layout, and Chapter 3 explained why there is no layout on the server.

“I just want it to run on mount”

The most common effect in any codebase, and usually the most wrong:

```
useEffect(() => {
  doTheThing(someValue)
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [])
```

The empty array plus the disable comment is a specific admission: *I know this effect reads things it doesn’t list, and I want it to run once anyway.*

The problem is that “once” isn’t a thing React promises. Effects re-run when their dependencies change; an empty array says “this effect depends on nothing,” and if that’s false, you’ve written a lie the compiler can’t catch. The effect keeps a snapshot of `someValue` from the first render, forever.

When you find yourself reaching for the disable comment, the useful question isn’t “how do I make the linter stop.” It’s **what am I actually trying to do**, and the answer is usually one of four things:

“Fetch data when the page loads.” RSC removed this. The server has the data before the component exists.

“Initialise something once per app.” Analytics, a global client, a polyfill. This probably isn’t component code at all — it’s module-level code that runs on import, or something set up outside React entirely. Putting it in a component ties app-wide setup to one component’s lifecycle for no reason.

“Set up a subscription.” Then the deps are correct as written and you should let them be. If it depends on `roomId`, it genuinely needs to re-run when `roomId` changes — that’s not the linter being annoying, that’s the linter being right.

“React to a change, but only sometimes.” This is the real case, and it’s what `useEffectEvent` above is for.

There’s also the ref-guard version:

```
const didRun = useRef(false)
useEffect(() => {
  if (didRun.current) return
  didRun.current = true
  doTheThing()
}, [])
```

This is usually written to suppress `StrictMode`’s double-invoke. It doesn’t fix anything — it hides the symptom that was telling you the setup/cleanup pair is asymmetric. If the effect genuinely can’t be run twice safely, it needs a cleanup, not a guard.

What RSC deleted

Two of the most common effects in a pre-RSC codebase no longer have a reason to exist.

Fetching on mount. Chapter 8’s example: `useEffect` fetching data and calling `setState`. The server does that now, before the component reaches the browser, without a loading state or a waterfall.

Syncing derived server data. All the effects that existed to keep a client copy of remote data current. Chapter 7 established that a mutation’s response contains the new tree, so there’s nothing to sync.

Strip those out and what’s left is genuinely small.

Effects that are still correct

Not zero. The rule holds — synchronise with something outside React:

- **Browser APIs.** Event listeners on window, IntersectionObserver, ResizeObserver, media queries, online/offline. Though for most of these, Chapter 8’s `useSyncExternalStore` is the better primitive.
- **Third-party widgets.** A map, a chart, an editor that owns its own DOM. Set it up, tear it down.
- **Live connections.** WebSockets, polling, server-sent events. Data that arrives because something else pushed it.
- **User-triggered client fetching.** A search-as-you-type box hitting an endpoint. Though “should this be in the URL” is worth asking first.
- **Measurement.** Read layout, adjust — usually `useLayoutEffect`.

Notice the shape. Every one of them involves a system that exists whether or not React does, and that has to be told when to start and stop.

That’s the whole job. Not “run this after render.” Keep that thing over there in step with this thing over here — and clean up after yourself when the relationship changes.

That’s the machine, end to end. A request arrives, the server runs your components, a payload comes back, the browser paints it,

React wakes up, mutations go the other way, and a small amount of genuinely local state and a smaller number of genuinely necessary effects handle the rest.

Every chapter so far has assumed one thing that isn't true: that none of this was cached. It was. Four different layers of it were, each with its own lifetime and its own way of going stale — and each capable of making the model you've just built look like it's lying to you.

References

- React docs — You Might Not Need an Effect, Synchronizing with Effects, `useEffectEvent`, `<Activity>`
- Dan Abramov — *A Complete Guide to `useEffect`*
- Dan Abramov — *Before You `memo()`*

Chapter 11 — Caching: Which Layer Is Lying to You

Every chapter so far has assumed nothing was cached. That was on purpose — you can't reason about an optimization until you understand the thing it optimizes. Now we turn it on.

This chapter is different in one way from the rest of the book. Everywhere else, you came in with a working model that was slightly wrong, and the job was to correct it. Caching isn't like that. Almost nobody has a wrong model of Next's caching — they have *no* model, just a revalidate they copied off Stack Overflow once and a vague dread when a page won't update.

So we're not going to correct anything. We're going to build the model from the ground up, starting with something you've definitely seen with your own eyes, and working down to the machinery underneath it.

I got tired of saying "it's probably a caching issue" every time something didn't update. That sentence is what you say when you have no model — it names a suspect without naming a layer. By the end of this chapter there are four layers, and you'll be able to name which one.

Start with what you've seen: the page that was already there

Navigate around a Next app with the Network tab open. Click into a product, hit back, click into another, hit back. Watch the requests.

Going *back* fires nothing. The previous page is just... there, instantly, including whatever you'd scrolled to or typed into it.

That’s a cache. The first one, and the one closest to you: **the browser is holding pages it already has**.

When you navigate with `<Link>`, Next keeps the RSC payload for routes you’ve visited — in memory, in the tab. Go back, and it replays the payload it already has instead of asking the server again. This is why back/forward is instant, and it’s the mechanism behind a claim from Chapter 3: layouts don’t re-render on navigation because the router is handing you a tree it kept, not fetching a new one.

Call this the **Router Cache**. It lives in the browser, it’s the shortest-lived of everything we’ll meet, and it’s the one you have the least control over.

It’s also the first liar. Here’s the trap, and you may have hit it: you change something, the change saves, the server has the new data — and the page still shows the old thing until you refresh. Nothing is broken on the server. The browser is just showing you a payload it’s still holding in memory. We’ll come back to how that gets cleared; for now, just note that “stale until refresh, then fine” has a specific cause, and it’s this layer.

So: one cache down, and we found it without reading a single doc. Just watched the Network tab.

Go one level down: the page that never hit your code

Now do a fresh load — type a URL, hit enter — on a marketing page or a blog post. Look at the timing. It’s fast in a way a database query can’t be, because no database was queried. No component of yours ran at all.

That page was built once, ahead of time, and saved. You’re being handed a file.

This is the **Full Route Cache** — the prerendered HTML and RSC payload from the build step we traced in Chapter 3. The `.html` and `.rsc` files on disk. For a route with no per-request data, that work happened once at build and every visitor gets the saved copy.

One layer up we had the browser holding pages it fetched. Here, one level deeper, the *server* is holding pages it built. Same idea — reuse instead of recompute — but a different place, a different lifetime, and a different way to clear it.

“But my content changes. I can’t build it once and freeze it forever.”

Right, and this is where most people’s understanding falls off, so we’ll go slowly.

You don’t have to choose between “built once, frozen” and “rebuilt on every request.” There’s a middle setting: build it, serve the saved copy, and **rebuild it in the background every so often** so it stays roughly fresh without ever making a user wait.

That middle setting is called ISR — Incremental Static Regeneration. Chapter 1 described its time-window behaviour and the scenarios it’s wrong for. What it didn’t say is that ISR is not the legacy pattern it’s often taken for. In Next 16 it was renamed and made much more capable, and it solves a problem you hit the moment your site is real: **you have more URLs than you can afford to build.**

Ten thousand products. You are not prerendering ten thousand pages on every deploy. So Next splits the work:

- Build the pages worth building ahead of time — the popular ones, listed in `generateStaticParams`.
- For every other URL, build a generic **App Shell** — the layout and frame, with a gap where the specific content goes.

Now a visitor asks for a product you *didn’t* prebuild. Instead of making them wait for a fresh render, Next hands over the App Shell **instantly** — frame on screen, skeleton where the product will be — then fills in the real product in the background. The *next* person who asks for that same URL gets the now-complete page from the cache. The route built itself, on demand, once, triggered by the first person who wanted it.

There's a structural rule that makes this possible, and it's the reason a page you've written might already be shaped this way without your knowing why:

```
export default function PropertyPage(
  props: PageProps<'/[id]')>
) {
  return (
    <Suspense fallback=<{<PropertySkeleton />>}>
      {/* await params INSIDE the boundary, not out here */}
      <PropertyBody params={props.params} />
    </Suspense>
  )
}
```

Don't await params at the top of the component. Pass the params promise down into a `<Suspense>` boundary and await it in there. Here's the reasoning, step by step: awaiting params at the top means the whole component depends on knowing which URL this is — so Next can't build anything until it knows. Awaiting it *inside* a boundary means everything above the boundary — the App Shell — can be built without knowing the URL at all. The shell becomes URL-independent, which is exactly what lets Next serve it instantly for a product it's never seen.

If you came from the Pages Router: this is `fallback: true`, `getStaticPaths` is now `generateStaticParams`, and `getStaticProps` with `revalidate` is now the `use cache + cacheLife` combo we're about to meet. The instinct to prebuild everything is worth resisting — build what's worth building and let the rest arrive on demand.

Go deeper: caching a piece instead of a whole page

The Full Route Cache is all-or-nothing per route. But most real pages aren't all-or-nothing. A property page has details that barely change (address, description, photos) sitting right next to a price that changes hourly. You can't freeze the whole route — the price would go stale — but you don't want to rebuild the whole thing every request either, when only the price is live.

So you need to cache *part* of a page. That's the next layer down, and it's the one you actually reach for by hand: the **Data Cache**, driven by the `use cache` directive.

```
async function getProperty(id: string) {
  'use cache'
  cacheLife('property')           // how long it stays fresh
  cacheTag(`property-${id}`)     // clear precisely, by id, later
  return getAllPropertyDetails(id)
}
```

`use cache` stores a result and reuses it **across requests** — the next visitor gets the stored copy instead of recomputing. That's the difference from the Full Route Cache above it: that one caches whole routes at build time, this one caches pieces, and can do it at runtime.

The one detail that will burn you on deploy

`use cache` has a default storage location, and on a serverless host — Vercel, most likely — **that location is the memory of one server instance, and it is not shared.**

Walk through what that means, because it's the single most expensive misunderstanding in the whole model. You cache your search results with `use cache`. Locally, one process, it works perfectly — second search is instant. You deploy. Now there are many instances. Request A caches its result in instance 1's

memory. Request B lands on instance 2, which has never seen it, and recomputes. Your cache “works” maybe a third of the time and you can’t figure out why.

The fix is one word:

```
async function getSearchResults(query: SearchQuery) {  
  'use cache: remote' // durable, shared across instances  
  cacheLife('search')  
  cacheTag('search')  
  return searchPropertiesApi(query)  
}
```

`use cache: remote` stores in the platform’s shared, durable cache instead of one instance’s memory. The rule: **anything built at build time can stay plain use cache (it’s a file, it’s durable); anything computed per-request that you want shared needs : remote.**

The same directive, moved up, does more

Here’s the part that turns `use cache` from a data cache into the main lever of the whole system. *What* it caches depends on *where* you put it:

- On a **function** → it caches the return value. The JSX that uses it still re-renders every time.
- On a **component** → it caches the *rendered output* — the actual RSC payload for that whole subtree. Every child render, every `.map`, every bit of formatting: done once, replayed.
- On a **file** → it caches the whole segment’s render.

Read that top to bottom and a strategy appears. You don’t push `use cache` *down* onto your data helpers to save a fetch. You push it *up* onto components and subtrees, so the server does the rendering work once and replays it. “Render as much on the server as possible” isn’t a slogan — it’s literally moving the directive up the tree.

And it composes, because of one rule: **a cached component's children and other JSX slots are passed *through* without entering its cache key.** So you cache the stable shell and thread the live parts through it:

```
async function CachedProperty({ id, pricingSlot }: {
  id: string
  pricingSlot: React.ReactNode // a live island, passed through
}) {
  'use cache'
  cacheLife('property')
  cacheTag(`property-${id}`)
  const property = await getProperty(id)
  return (
    <PropertyContent
      property={property}
      pricingSlot={pricingSlot}
    />
  )
}
```

The property render is cached for a long time and cleared by tag. The pricingSlot — a separate `<Suspense>` island passed in from above — streams fresh on its own short cycle. One cached, one live, interleaved in a single tree. That works only because props pass through as slots without becoming part of the key. Imports would pull the code in; props don't. That's the same boundary distinction from Chapter 5, showing up again at a different layer.

Two rules that will each cost you a debugging session if you don't know them:

- **A cached component's props are its cache key, and they're serialized with the strict RSC rules** — primitives and plain objects only. Pass a Date or a class instance or a function as a keying prop and it breaks. Pass-through children and slots are exempt — they're not keys.

- **Keep `notFound()` outside a cached component.** Return null from the cached part; throw not-found in the caller.

The bottom layer: the same read, many times, one render

We've gone from the browser, to the built route, to the cached piece. One layer left, and it's the smallest and lowest.

Inside a single render, the same data often gets read more than once. Your layout needs the current user. So does the page. So does `generateMetadata`. Three components, three calls to `getUser()`, and naively, three database hits for one page load.

The bottom layer collapses those into one. It's React's cache:

```
import { cache } from 'react'

const getUser = cache(async (id: string) => {
  return db.users.findById(id)
})
```

Now `getUser(1)` runs its body **once per render**, no matter how many components call it. The rest get the stored result.

This is the layer most likely to be confused with the one above it, so here's the sharp line. The docs describe `react cache` as “a `useMemo` at the function level” — and that comparison is the trap, because it makes it sound persistent. It isn't. **Its entire world is one render, one request.** When the request ends, the table is gone. It saves work *within* a single page load; it saves nothing for the next visitor. That's `use cache`'s job, one layer up. Same word, completely different lifetime — which, by now, is the most predictable thing in this whole framework.

Two things that bite:

- It **caches errors too** — if the read throws, every caller in that render re-throws it.
- **Each cache() call is its own cache** — wrap the function once, export it, import that everywhere. `cache(fn)` in two files gives you two unrelated caches.

One companion piece, new in React 19.2: `cacheSignal()`. When you `cache()` an expensive fetch, `cacheSignal()` gives you an abort signal that fires when *that render's* lifetime ends — render finished, aborted, or dropped. Hand it to `fetch`, and if React abandons the render (user navigated away mid-stream), the fetch aborts instead of running to completion for a page no one will see.

```
import { cache, cacheSignal } from 'react'

const getChartData = cache(async (id: string) => {
  const signal = cacheSignal()
  const res = await fetch(`/api/chart/${id}`, { signal })
  return res.json()
})
```

The switch that turns caching on — and the error it introduces

One piece of plumbing connects all of this, and you have to opt in:

```
// next.config.ts
const nextConfig = { cacheComponents: true }
```

Flipping that flag inverts the default. Without it, Next tried to guess what to cache. With it, **everything is dynamic — runs per request — unless you either prerender it or wrap it in `use cache`**. No more guessing; you decide, explicitly.

The catch, and you'll meet it on your first build: any per-request data — `cookies()`, `headers()`, `searchParams`, an uncached `fetch` — that is *not* inside a `<Suspense>` boundary becomes a **build error**. Next is refusing to guess whether that page is static or dynamic; you have to say.

The tool for saying “this part is dynamic, on purpose” is `connection()`:

```
import { Suspense } from 'react'
import { connection } from 'next/server'

async function LiveBanner() {
  // everything after this line is genuinely per-request
  await connection()
  const now = Date.now()
  return now > promoEnd ? <Normal /> : <Promo />
}

export default function Page() {
  return (
    <Suspense fallback=<BannerSkeleton />>
      <LiveBanner />
    </Suspense>
  )
}
```

The first place this bites is time. Under `cacheComponents`, `Date.now()` or `new Date()` before any other dynamic read throws a build error — because reading the clock during a prerender would bake the *build* time into a supposedly static page. Three ways out, best first: move it to a Client Component if it's for display; wrap it in `use cache` if a build-time value is actually fine; or `await connection()` inside a `Suspense` boundary if it genuinely must be per-request.

Now put all four together

We built them bottom-up as we found them. Here they are as one picture, and now the names have meaning instead of being jargon:

Layer	What it holds	Where it lives	Lifetime	How you clear it
Router Cache	RSC payloads for visited routes	The browser tab	Shortest — minutes, or until reload	A mutation returning a fresh tree, or a hard navigation
Full Route Cache	Whole prerendered routes (.html + .rsc)	Server disk	Until deploy, a time window, or an on-demand clear	Deploy, cacheLife, or tag/path revalidation
Data Cache (use cache)	A computed value or a rendered subtree	Instance memory, or shared with : remote	Across requests, as long as cacheLife allows	By time, or precisely by tag
Request Memoization (react cache)	One read, reused within one render	Server memory, per request	One render. Gone when the request ends	You don't — it dies on its own

Four caches, four lifetimes, four ways to clear them. Which brings us to clearing.

Invalidation, one story per layer

“How do I clear the cache” has four answers, and using the wrong one is why a cache feels haunted:

- **Request Memoization** — you don't clear it; it's gone when the render ends. If it feels stale, your tree fetches the same thing in two separate render trees — consolidate.
- **Data Cache** — by time (`cacheLife`) or by tag. Tags are the precise tool: a price change fires one tag and touches nothing else.
- **Full Route Cache** — on deploy, on a time window, or on-demand by tag/path.
- **Router Cache** — the shortest-lived; a mutation that returns a fresh tree, or a hard navigation, clears it.

The tag mechanism is most of a real invalidation strategy. Tag caches by the thing that changes, and clear exactly that when it changes:

```
// app/api/revalidate/route.ts – guard it with a secret
export async function POST(req: Request) {
  const { tags } = await req.json()
  for (const tag of tags) revalidateTag(tag, 'max')
  return Response.json({ revalidated: tags })
}
```

Two revalidation functions, and picking wrong is a common mistake: `revalidateTag(tag, 'max')` in route handlers — the bare single-argument form is deprecated and errors in TypeScript now, so pass `'max'` or a `cacheLife` profile — and `updateTag()` inside Server Actions. Tag by `property-<id>`, `pricing-<id>`, `search`, and each clears independently, which is what makes a long `cacheLife` safe. The duration is just a backstop; correctness comes from firing the tag on the change.

Which layer is lying to you

You now have enough to debug caching by symptom instead of by guessing. A page shows something wrong — walk it:

Stale after a data change? → Data Cache or Full Route Cache. Did the mutation fire the right tag? Is the data behind use cache with a long life and no tag? Then it's stale by design until the window passes.

Worked locally, broken on the deploy? → Data Cache, the serverless memory trap. Plain use cache on a runtime value. Switch to use cache: remote.

Stale only until you refresh, then fine? → Router Cache. The server is right; the browser is replaying a payload it kept. A mutation returning a fresh tree, or a reload, clears it.

Same query hitting the database several times in one render? → Request Memoization. Wrap the reader in react cache, import the one wrapped copy.

Build failed with a dynamic-data error? → Not a lie, a requirement. Uncached request data outside <Suspense> under cacheComponents. Wrap it, cache it, or connection() it.

Four layers, four failure signatures. The fix was never the hard part. Knowing which of the four you were looking at was — and now you do, because you watched each one appear instead of being handed a list.

References

- Next.js — Caching with Cache Components; ISR with Cache Components
- Next.js — use cache, use cache: remote, cacheLife, cacheTag, connection
- React — cache, cacheSignal

Chapter 12 — The Map

Eleven chapters. One request.

This is the short one, and it's the one to come back to. Everything below is the full lifecycle in order, each step naming the chapter that explains it.

You already knew most of this. You've shipped React for years. But knowing the parts isn't the same as knowing the schedule — and that gap matters more now that a lot of code arrives already written, and the job is knowing whether it's right.

The lifecycle

You type a URL and hit enter.

The request reaches your server. Nothing in the browser has executed. One thing on the server has: `proxy.ts` runs before routing on every request — the single exception to everything below. → *What Runs When, and What Comes Out*, appendix

Something decides whether to run your code at all.

If the route was prerendered, two files come off disk and no component executes. If it wasn't, everything below happens now, per request. → *What Runs When*

Your Server Components execute.

In Node. They query databases, read files, hold secrets. They run once and are finished. Their code never enters a browser bundle. → *What Runs When*

They produce a payload, not HTML.

A text description of a component tree. Where a client component was, there's a hole with an address on it. → *What Runs When*

The address came from your bundler.

Which resolved your imports, transformed your TypeScript, walked your module graph, split it into chunks, and assigned every module a stable ID. → *What Your Bundler Actually Does*

And there were two graphs, not one.

A server graph and a client graph, built from one repo, with "use client" as the boundary between them and a manifest as the phone book. → *Two Graphs, One Repo*

A second renderer turns the payload into HTML.

It resolves each address, loads the real client component, and executes it — on the server. Producing a string. No CSS is evaluated, no layout is computed, nothing is painted. → *What Runs When*

Parts of the tree aren't ready, so they're sent later.

Suspense boundaries let the response stay open. The shell ships, and missing pieces arrive as more rows on the same connection. → *Suspense Is Not a Loading Spinner*

The browser paints. Nothing is interactive.

HTML on screen, no React yet. Late-arriving content still fills in, patched by a small inline script that runs before hydration. → *Rendering Patterns, Suspense Is Not a Loading Spinner*

JavaScript arrives and React wakes up.

It reads the payload, reconstructs the tree, resolves the same addresses against a different manifest, and executes your client components a second time — attaching handlers to DOM that already exists. → *What Runs When*

Which is reconciliation, doing what it always did.

Elements, keys, diffing, the commit phase. RSC changed where elements come from, not what React does with them. → *How React Actually Updates the DOM*

The user clicks a link.

Server Components for the new route execute. A payload comes back. No HTML at all. React diffs the new tree against the old one, and layouts that didn't change don't re-render. → *What Runs When*

The user submits a form.

A POST to a generated endpoint, addressed by an ID the bundler assigned. The function runs on your server. The response is a new

tree, not JSON — so the mutation and the refetch are one round trip.
→ *A Mutation, End to End*

And most of that work didn't need to happen twice.

Four caches sit under everything above — one in the browser holding routes you've visited, one holding whole routes built ahead of time, one holding pieces you marked yourself, one collapsing repeated reads inside a single render. Each has its own lifetime and its own way of going stale. → *Caching: Which Layer Is Lying to You*

The user clicks a button.

State changes. React re-renders a subtree and mutates the DOM. Nothing leaves the machine. This is the only moment that is ordinary React. → *What Runs When*

And that state was smaller than you expected.

Because the data belongs to the server, anything shareable belongs to the URL, and form values belong to the form. What's left is local and ephemeral. → *What's Actually Left for the Client, What Redux Was Actually For*

Effects synchronise the rest.

Not lifecycle hooks. Setup and cleanup pairs, keeping something outside React in step with something inside it. → *Effects Are Not Lifecycle*

Reading order

Start here if you want the shape: **What Runs When, and What Comes Out**. Everything else is a zoom into one of its steps.

Start here if you want the build: **What Your Bundler Actually Does**, then **Two Graphs, One Repo**.

Start here if you're debugging something specific: whichever step above matches your symptom.

Two chapters sit outside the numbered lifecycle. **What Redux Was Actually For** is a companion to *What's Actually Left for the Client* — for the applications where the answer genuinely is “a lot

of client state,” and for untangling what a store was ever solving. And **Caching: Which Layer Is Lying to You** is the one to read last whatever order you take the rest in, because everything before it assumes caching is off.

Deliberately not here: prefetching, the router hooks, parallel and intercepting routes, scroll restoration. Navigation internals are a browser story more than a React one, and they get the treatment they deserve in the next book.

The pattern underneath

One thing worth naming, having got here.

Almost every confusion this book untangled came from a name that implied the wrong thing.

Render suggested pixels; it means calling a function. *"use client"* suggested a location; it means a boundary. *Suspense* suggested a spinner; it means a transport guarantee. *SSG* and *Static* suggested two strategies; they're one mechanism counted differently. *Server-rendered* suggested HTML; most page views never produce any. *Middleware* suggested a place to do work; it's a proxy that should mostly get out of the way. *Cache* suggested one thing; it's four, with four lifetimes and one shared word.

The tools are honest about what they do. The vocabulary isn't — and it was mostly inherited from a version of the framework that no longer exists.

Which is a reasonable definition of what it means to actually know a system: not that you memorised the API, but that you stopped being misled by what things are called.

Glossary

Terms as this book uses them, with the chapter that explains each properly.

App Shell — A route's layout and frame, built without knowing which URL it will serve. Lets Next hand a visitor something instantly for a page it has never rendered. *Ch 11*

Chunk — One output file a bundler emits, containing many modules. Its filename carries a content hash, which is what makes browser caching work. *Ch 4*

Chunk group — The set of chunks loaded together for one entry point. Turbopack only merges chunks within a group, which is what stops a merge from shipping one route's code to another. *Ch 4*

Client Component — A component in a module marked "use client". Runs on the server during the SSR pass *and* again in the browser at hydration. *Ch 3*

Client manifest — A table the bundler writes at build time, mapping module IDs to the chunks holding them. Read once by the SSR pass and once by the browser. *Ch 5*

Commit phase — The step where React applies its computed changes to the real DOM, in one synchronous pass. *Ch 2*

Concurrent rendering — Multiple renders in flight at once, with only the latest committing. Enabled by Fiber's interruptibility. *Ch 2*

CSR (Client-Side Rendering) — The browser receives an empty shell and builds the entire DOM from JavaScript. *Ch 1*

Data Cache — The layer driven by use cache. Stores a computed value or a rendered subtree and reuses it across requests. *Ch 11*

Deps array — The second argument to useEffect. A contract stating which values the effect stays in sync with — not a performance knob. *Ch 10*

Double buffering — React's technique of holding two Fiber

trees, current and workInProgress, so a render can be paused or abandoned without leaving the UI half-updated. *Ch 2*

Entry point — A place a bundler starts walking the graph. Routes are server-graph entries; every "use client" module is a client-graph entry. *Ch 4, 5*

Export condition — A key in a package's exports map that selects a different file per environment — import, browser, node, and critically react-server. *Ch 4, 5*

FCP (First Contentful Paint) — When the user first sees anything that isn't a blank screen. *Ch 1*

Fiber — React's unit of work, one per component instance, and the architecture that made reconciliation interruptible. Also where useState values live. *Ch 2, 8*

Flight — React's wire format for describing a component tree. What this book calls the RSC payload. *Ch 3*

Full Route Cache — Whole prerendered routes held on disk as .html and .rsc files. *Ch 11*

Hydration — React executing client components in the browser and attaching event handlers to DOM that the server already produced. It adopts existing DOM rather than creating it. *Ch 3*

Hydration gap — The window where a page looks interactive but isn't, because JavaScript hasn't finished. *Ch 1*

Hydration mismatch — The server render and the browser render disagreed. In production React silently discards the server HTML for that subtree and re-renders it. *Ch 2*

ISR (Incremental Static Regeneration) — Serving a saved copy while rebuilding in the background, plus — in Next 16 — building routes on demand that were never prebuilt. *Ch 1, 11*

Key — The prop that gives a list item stable identity across renders. Changing a key deliberately is also the correct way to reset a component's state. *Ch 2, 10*

Minification — A syntax-level pass that renames variables and strips whitespace, after chunking. Not tree shaking. *Ch 4*

Module graph — Modules as nodes, import statements as edges. Buildable ahead of time only because ESM imports are static. *Ch 4*

Module ID — A module’s deterministic identity inside a chunk. Deterministic so that a one-line change doesn’t invalidate caches across your whole app. *Ch 4*

PPR (Partial Prerendering) — Static and dynamic content on one page, default behaviour in Next 16. *Ch 1*

Proxy (proxy.ts) — The file that runs on every request before routing. Renamed from `middleware.ts` in Next 16, and the rename is a correction: direct traffic, don’t do work. *Ch 3, appendix*

Reconciliation — Comparing a new tree against the old one to find the minimum set of DOM changes. $O(n)$ only because of the type and key assumptions. *Ch 2*

Render — Three different things. React’s: call a function, get an object. The browser’s: turn markup into pixels. Server-side rendering’s: produce an HTML string. The server does the first and third, never the second. *Ch 3*

Render phase — The step where React calls your component functions to produce a new tree. Nothing has touched the DOM yet. *Ch 2*

Request Memoization — React’s `cache()`. Collapses repeated reads of the same data within a single render. Dies when the request ends. *Ch 11*

RPC (Remote Procedure Call) — Making a call to code on another machine look like a local function call. Server Actions are RPC without a schema. *Ch 7*

RSC (React Server Components) — Components that execute only on the server and never ship their code to the browser. *Ch 1, 3*

RSC payload — A text description of a component tree, with addresses where client components go. Produced first; HTML is generated from it. *Ch 3*

Router Cache — RSC payloads for visited routes, held in the browser tab. Why back and forward are instant, and why a page can look stale until you refresh. *Ch 11*

Selective hydration — React hydrating boundary by boundary, and jumping the queue to hydrate whichever boundary the user just clicked. *Ch 1, 6*

Server Action — A function marked "use server", callable from the browser through a generated endpoint. A public endpoint you didn't write a route for. *Ch 7*

Server Component — A component that runs once, on the server, and is finished. Its code never enters a browser bundle. *Ch 3*

server-only — A package that resolves to an empty module in the server graph and a throwing module anywhere else, so a leak fails at build time. *Ch 5*

Signals — Fine-grained reactivity: a value change updates exactly the DOM nodes bound to it, with no VDOM and no diffing. *Ch 2*

Source map — The lookup table from minified output back to your source. Without it a production stack trace is useless. *Ch 4*

SSG (Static Site Generation) — Rendering at build time and serving saved files. In Next 16's build output, **•** rather than **◦** just means the route had dynamic segments you enumerated. *Ch 1, 3*

SSR (Server-Side Rendering) — Producing HTML on the server so the browser can paint before JavaScript arrives. *Ch 1*

SSR pass — The second render on the server, which consumes the payload and produces HTML — executing your client components in the process. *Ch 3*

Stale-while-revalidate — Serve the saved copy now, rebuild in the background. The request that triggers revalidation still gets stale data. *Ch 1*

StrictMode — Development-only double-invoking of effects. A test of whether your setup/cleanup pair is symmetric, not a nuisance to suppress. *Ch 10*

Suspense boundary — A declaration that a subtree may arrive late. A transport primitive, not a loading-state API — and also a hydration unit. *Ch 6*

Tearing — Components in one render pass reading different values from the same external source. What `useSyncExternalStore` exists to prevent. *Ch 8*

Tree shaking — Dropping modules and exports nothing

reaches, using the module graph. Defeated by barrel files and by modules with side effects. *Ch 4*

TTI (Time to Interactive) — When the page actually responds to input. Can lag FCP by seconds. *Ch 1*

Turbopack — The bundler, default for both `next dev` and `next build` since Next 16. Keeps one logical graph across multiple environments. *Ch 4*

use cache — The directive that puts a function's return value, a component's rendered output, or a whole segment into the Data Cache. What it caches depends on where you put it. *Ch 11*

use cache: remote — The same directive, stored in the platform's shared durable cache instead of one instance's memory. The fix for "worked locally, broke on deploy." *Ch 11*

"use client" — Marks the root of a client subtree. A boundary, not a location — everything reachable from it enters the client graph. *Ch 3, 5*

"use server" — Marks a function as a Server Action. The bundler replaces its body with a stub carrying an ID. *Ch 7*

VDOM (Virtual DOM) — A plain object tree describing the UI. A developer-experience innovation with acceptable performance, not a performance innovation. *Ch 2*

Index

References are to chapters, not pages, so this index works the same in print and on a reader. Bold marks the chapter where a term is explained rather than merely used.

A

<Activity> — **10**

App Shell — **11**

B

barrel files — 4, **5**

C

cacheComponents — **11**

cacheLife — **11**

cacheSignal — **11**

cacheTag — **11**

chunking — 4

client graph — 5

commit phase — 2

concurrent rendering — 2

connection() — **11**

Context — **8**, 9

CSR — 1

D

Data Cache — **11**

deps array — **10**

double buffering — 2

E

error boundaries — **6**

export conditions — 4, **5**

F

FCP — 1

Fiber — 2, 8

Flight — 7

FormData — 7

Full Route Cache — **11**

G

generateStaticParams — **3**, 11

H

hydration — 1, 2, **3**, 6, 7, 8

hydration gap — **1**

hydration mismatch — **2**, 3, 8

I

ISR — **1**, 11

K

keys — **2**, 5

L

loading.tsx — **6**

M

memo — **2**, 8

minification — **4**

module graph — **4**

module IDs — **4**

P

PPR — **1**

prerendering — **3**, 11

proxy.ts — **3**

R

reconciliation — **2**, 3, 6, 7

render phase — **2**

Request Memoization — **11**

revalidateTag — 1, 7, **11**

Router Cache — **11**

RPC — 7

RSC payload — 1, **3**, 4, 5, 11, 12

S

searchParams — **8**

Server Actions — 3, 7

Server Components — 1, **3**, 5, 6, 12

server-only — **5**

Signals — **2**

source maps — **4**

SSG — **1**, 3

SSR pass — **3**, 8

startTransition — **2**

streaming — 3, **6**, 11

StrictMode — **10**

Suspense — 1, 2, **6**, 11, 12

T

tearing — **8**

TTI — **1**

Turbopack — **4**

U

use cache — 1, **11**

"use client" — 1, 3, **5**, 7, 8, 12

"use server" — **7**

useActionState — **7**

useCallback — **2**

useEffect — 8, **10**

useEffectEvent — **10**

useLayoutEffect — **10**

useMemo — **2**

useOptimistic — **7**

useState — 1, 5, **8**

useSyncExternalStore — **8**

V

VDOM — 1, **2**

About the Author

Mayank Jhunjhunwala is a senior software engineer. He builds software — web and mobile — from scratch, and has several products running in production.

He'd like to work at a YC company in the near future, and to lead a YC startup in a more distant one.

More than anything else, he loves storytelling, across every genre — especially comics and games.

Web — manku27.dev

LinkedIn — linkedin.com/in/mayankc137

Email — jjmayank98@gmail.com

If this helped, or if something here is wrong, he'd like to hear about it.